

Title: A Unified Executors Proposal for C++

Authors: Jared Hoberock, jhoberock@nvidia.com
Michael Garland, mgarland@nvidia.com
Chris Kohlhoff, chris@kohlhoff.com
Chris Mysisen, mysen@google.com
Carter Edwards, hcedwar@sandia.gov
Gordon Brown, gordon@codeplay.com

Other Contributors: Hans Boehm, hboehm@google.com
Thomas Heller, thom.heller@gmail.com
Lee Howes, lwh@fb.com
Bryce Leibel, bryceleibel@gmail.com
Hartmut Kaiser, hartmut.kaiser@gmail.com
Bryce Leibel, bryceleibel@gmail.com
Gor Nishanov, gorn@microsoft.com
Thomas Rodgers, rodgert@twrogers.com
David Hollman, dshollm@sandia.gov
Michael Wong, michael@codeplay.com

Document Number: P0443R1

Date: 2017-01-06

Audience: SG1 - Concurrency and Parallelism

Reply-to: jhoberock@nvidia.com

1 Introduction

This paper describes a programming model for executors, which are modular components for creating execution. Executors decouple software from the disparate, non-uniform interfaces of execution resources and present a uniform interface for work creation. The model proposed by this paper represents what we believe is the functionality necessary to compose executors with existing standard control structures such as `std::async()` and parallel algorithms, as well as near-standards such as the functionality found in various technical specifications, including the Concurrency, Networking, and Parallelism TSes. We intend this proposal to be an extensible basis for future development of execution in C++ which necessarily entails functionality beyond the scope of this basic proposal.

In our programming model, executors introduce a uniform interface for creating execution that may not be common to the underlying execution resources actually responsible for the mechanics of implementing that execution. There are three major concepts involved in this interplay: execution resources, execution contexts, and executors.

An **execution resource** is an instance of a hardware and/or software facility capable of executing a callable function object. Different resources may offer a broad array of functionality and semantics, and may range from SIMD vector units accessible in a single thread to an entire runtime managing a large collection of threads. In practice, different resources may also exhibit different performance characteristics of interest to the performance-conscious programmer. For example, an implementation might expose different processor cores, with potentially non-uniform access to memory, as separate resources to enable programmers to reason about locality.

An **execution context** is a program object that represents a specific collection of execution resources.

An **executor** is an object associated with a specific execution context. It provides a mechanism for creating execution agents from a callable function object. The agents created are bound to the executor's context, and hence to one or more of the resources that context represents.

Executors themselves are the primary concern of our design.

1.1 Design goals

The design outlined in this proposal is intended to achieve our goals for executors, which are objects for creating all kinds of execution in C++. Executors should be *composable*, *adaptable*, and *customizable*, which we believe reflects the needs of users, library implementors, and executor authors, respectively. Short code examples demonstrating how our design accommodates the needs of these different audiences may be found in the [Appendix](#).

By *composable*, we mean that executors should cross software boundaries allowing disparate software components to interoperate using a commonly-understood protocol. By *adaptable*, we mean that executors specially-designed for a particular use case should be applicable in other use cases when it is possible. By *customizable*, we mean that requirements placed on executors should be broad enough to encompass user-defined executors, rather than limited to the set of concrete executors which may eventually become part of the C++ Standard Library. Only a design including well-defined interfaces with clear semantics will achieve these goals.

1.2 Customization Points

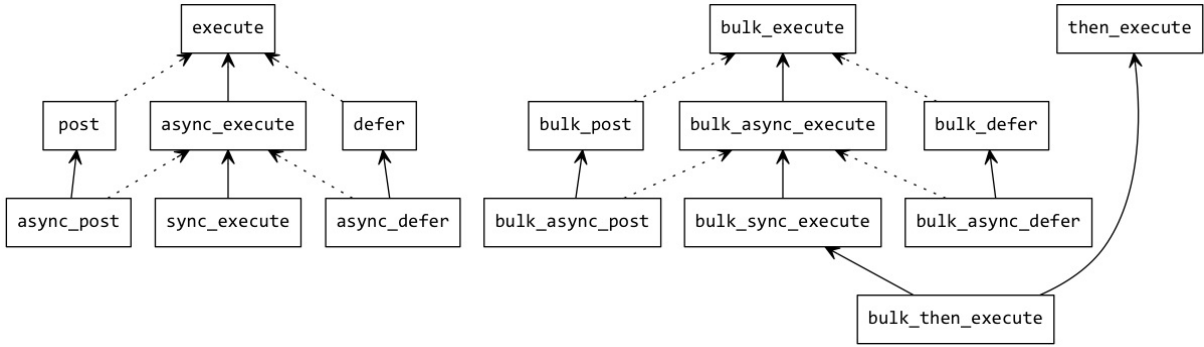
In our design, these well-defined interfaces with clearly-expressed execution semantics are called executor *customization points* and are foundational to the mechanics of our design. Each executor customization point is a function which delineates a specific use case which we have identified as fundamental to the needs of the C++ Standard Library and other technical specifications for execution creation. For example, the customization point `execution::async_execute()` asynchronously creates a single execution agent to invoke a given function and returns a future corresponding to that function's eventual result. A C++ Standard Library function like `std::async()` may compose with a user-supplied executor by using calling `execute::async_execute()` in its implementation. As another example, the customization point `execution::sync_execute()` synchronously creates a single execution agent to invoke a given function and immediately returns that function's result. These semantics provide a natural fit for the composition of user-provided executors with `std::invoke()`.

Because we envision executors to be the workhorses of execution in generic code, it is critical that they be adaptable to a variety of use cases. For example, a generic function like `std::invoke()` should be interoperable with as many types of executor as possible, not just that set of executors which natively provide the synchronous, single-agent execution function `sync_execute()`. To generalize across use cases, we have designed executor customization points to adapt the behavior of an executor when its native behavior is not a precise match for the use case of interest. For example, `std::invoke()` may interoperate with an executor which natively provides the execution function `async_execute()` by calling the customization point `execution::sync_execute()`. This customization point will adapt the executor by calling its natively provided `async_execute()` execution function, and wait on the resulting future. In this way, generic code may uniformly compose with executors with minimal restriction.

These adaptations performed by executor customization points are always performed in such a way as to ensure that various semantic guarantees provided by executors are never weakened. For example, if an executor guarantees that its possibly-blocking execution functions are always non-blocking, then an adapted implementation of `execution::async_execute` will always be non-blocking when that executor is composed with this customization point.

Our design focuses on defining a set of optional execution functions instead of imposing a set of strict executor requirements to maximize the latitude of executor authors. Rather than prematurely attempt to circumscribe the universe of all possible executors with a set of universal requirements, we believe it is more flexible to identify a set of use cases for work creation which can grow in the future. Executor authors may choose to natively support one or more of these use cases by implementing the appropriate execution function, and may opt in to natively supporting new use cases as they are introduced. Due to their adaptability, an executor will be future-proof to new customization points.

The following diagram shows the automatic adaptations performed by the customization points, assuming a corresponding native implementation (member or free function) is unavailable. A dotted line shows an adaptation that will be used only if `executor_execute_blocking_category_t<Executor>` is `non_blocking_execution_tag`.



1.3 Executor type traits

In order to help programmers define their own ad hoc requirements for executor types, our design provides a set of executor type traits for introspecting the properties of executors at compile time. These fine-grained type traits detect characteristics such as native support for execution functions as well as compatibility between an executor type and an executor customization point.

For example, for an execution function such as `execute`, the executor supports the following traits:

- `has_execute_member<X>` and `has_execute_member_v<X>`, to detect the presence of a member named `execute` that satisfies the corresponding syntactic requirements.
- `has_execute_free_function<X>` and `has_execute_free_function_v<X>`, to detect the presence of a free function named `execute`, accepting the executor as the initial parameter, and that satisfies the corresponding syntactic requirements.
- `can_execute<X>` and `can_execute_v<X>`, to determine whether the corresponding customization point is well-formed for the executor.

Programmers may employ these type traits at software boundaries to define requirements for executor composition as well as reject types of executors which are known to be incompatible.

These traits may also be used to determine which implementation strategy will be used by a customization point. That is, whether the customization support can use native support (via member or free function) or an adaptation. If a particular customization point's adaptation is inefficient when implementing some higher level algorithm, the trait may be used to select an alternative algorithm implementation not based on that customization point.

While these traits are not strictly required by this proposal -- we may employ SFINAE to detect the presence of members or free functions -- correctly detecting adherence to the syntactic requirements is complex. Therefore these traits are included to simplify feature detection for programmers, as well as reducing the verbosity of this proposal's wording.

1.4 Executor Categories

Our design organizes executors into *executor categories* based on how they are expected to be used by software components composing with them. These categories correspond to the application domains served by the C++ Standard Library and the Parallelism, Concurrency, and Networking Technical Specifications. The requirements for membership in an executor category include basic requirements on executor types as well as requirements based on their compatibility with customization points. For example, the executor category `TwoWayExecutor` includes all executors compatible with the two-way, single-agent executor customization points such as `execution::async_execute()`. Executor categories are not necessarily mutually exclusive, and one executor type may be a member of multiple categories if the requirements for those categories admit mutual membership.

This proposal defines four named categories: `OneWayExecutor`, `TwoWayExecutor`, `BulkTwoWayExecutor`, and `NonblockingOneWayExecutor`. `OneWayExecutor` and `TwoWayExecutor` summarize the requirements for basic executors which create single-agent execution with minimal guarantees. `OneWayExecutors` do not provide a channel for synchronizing with the completion of execution, while `TwoWayExecutors` do provide such a channel. `BulkTwoWayExecutor` summarizes requirements for executors which create bulk-agent execution and also provide a channel for synchronization. This category summarizes the requirements for executors which can compose with parallel algorithms. Finally, `NonblockingOneWayExecutor` extends `OneWayExecutor`'s requirements by demanding that an executor's execution functions do not block the calling thread of those operations. This additional requirement on blocking behavior is critical to the needs of the Networking TS.

We have not attempted to completely capture every kind of interesting collection of executors in our categorization. By design, our categorization of executors is incomplete to accommodate future extension. Moreover, we recognize that the particular categories we define may not precisely match the requirements of all existing software interfaces which need to compose with executors. For example, the regularity of our design suggests the existence of a hypothetical executor category named `NonblockingHostBasedBulkTwoWayExecutor`. Such a category would correspond to executor types which create bulk, two-way execution hosted on a thread, and would create execution in a way that is guaranteed not to block the calling thread. However, our proposal does not define such a category. Instead, this category could be created by composing the executor type traits described above.

1.5 Execution Contexts

In our design, *execution contexts* are objects that represent a specific collection of resources and may be used by executor to create execution. For example, a thread pool is an execution context which manages a collection of threads upon which an associated executor may create execution agents. For our purposes of defining a programming model for executors, the only salient expectation for execution contexts is the ability to obtain an executor from them. However, we have chosen not to prescribe a required interface for doing so. Instead, we have defined very basic requirements for execution contexts which allow almost any type of object to be an execution context. Our requirements for execution contexts are deliberately minimal and intended to provide a standard protocol for obtaining an executor's context and reason about its identity in generic code. Yet, these minimal requirements still allow for further refinement in future libraries. For example, we illustrate requirements for a `NetworkingExecutionContext` that we expect the Networking TS to define which would enumerate expectations on execution contexts compatible with networking use cases. This refinement's requirements significantly strengthen our weak requirements in part by demanding that all such contexts derive from a specific concrete class.

In order to evaluate this proposal, it will be useful to have at least one concrete execution context readily available. Our proposal specifies a single execution context, `static_thread_pool`, which abstracts a explicitly-sized collection of threads and implements

an effectively unbounded work queue. This thread pool type provides functions for typical thread pool operations as well as an `.executor()` function for obtaining an executor associated with the thread pool. We recognize that different types of thread pools are suited to different use cases and emphasize that this proposal's `static_thread_pool` represents a single design in this space which is not intended as definitive.

1.6 Extensibility

This proposal is intended to provide a foundation for future enhancements to executors and execution in general in C++. Naturally, it is incomplete and we anticipate extension. Here, we discuss briefly our vision for how the major components of our overall design can be extended by future follow-on proposals.

Future concept. One immediate concern is the conceptualization of `std::future`-like types. We anticipate that some types of executors will expose idiosyncratic, custom, and non-standard future types to represent asynchronous tasks due to requirements of underlying execution resources. An elaboration of the requirements of a hypothetical `Future` concept is needed.

Execution resources. Our proposal is silent about how a program might enumerate and represent the available execution resources present in the system. A future proposal describing a means of representing system resources with standard execution contexts and obtaining executors from them would provide programmers concerned with performance the tools to reason about locality.

Execution contexts. This paper proposes a single concrete execution context, `static_thread_pool`, which embodies one approach for representing a thread pool. There are other approaches to thread pools with different features and limitations which others could propose. For example, a hypothetical `dynamic_thread_pool` type could automatically change its thread count to adapt to the state of the system, with the goal of guaranteeing concurrent execution. Another possible execution context could emulate the existing behavior of `std::async()` to aid in migration from the standard library's existing features for concurrency and parallelism to this new model of executors. Such a context would allow programmers to introduce executors without breaking any assumed semantics of `std::async()`, such as concurrent execution agents, thread-per-request, and future blocking behavior.

Besides introducing concrete execution context types, future proposals could refine our `ExecutionContext` concept by introducing concepts with additional requirements. As an example, a parallel execution framework might define requirements for a hypothetical `IntrospectableExecutionContext` which would require contexts to provide functionality for resource introspection. Generic code could depend on such functionality to query the number of hardware threads associated with execution context. These and other extensions to our basic model of execution contexts may be explored as future proposals.

Executor categories. Future proposals might expand our executor categorization to additional application domains if the categorizations proposed here are insufficient for representing their requirements. For example, our proposal does not directly address issues related to heterogeneous execution resources. These issues can be addressed in the future with new kinds of executors incorporated into the overall framework with the appropriate categorization.

1.7 Changelog

1.7.1 Changes since R0

- Executor category simplification
- Specified executor customization points in detail
- Introduced new fine-grained executor type traits
 - Detectors for execution functions
 - Traits for introspecting cross-cutting concerns
 - Introspection of mapping of agents to threads
 - Introspection of execution function blocking behavior
- Allocator support for single agent execution functions
- Renamed `thread_pool` to `static_thread_pool`
- New introduction

2 Proposed Wording

2.0.1 Header <execution> synopsis

```
namespace std {
namespace experimental {
inline namespace concurrency_v2 {
namespace execution {
```

```
// Member detection type traits:
```

```
template<class T> struct has_execute_member;  
template<class T> struct has_post_member;  
template<class T> struct has_defer_member;  
template<class T> struct has_sync_execute_member;  
template<class T> struct has_async_execute_member;  
template<class T> struct has_async_post_member;  
template<class T> struct has_async_defer_member;  
template<class T> struct has_then_execute_member;  
template<class T> struct has_bulk_execute_member;  
template<class T> struct has_bulk_post_member;  
template<class T> struct has_bulk_defer_member;  
template<class T> struct has_bulk_sync_execute_member;  
template<class T> struct has_bulk_async_execute_member;  
template<class T> struct has_bulk_async_post_member;  
template<class T> struct has_bulk_async_defer_member;  
template<class T> struct has_bulk_then_execute_member;
```

```
template<class T> constexpr bool has_execute_member_v = has_execute_member<T>::value;  
template<class T> constexpr bool has_post_member_v = has_post_member<T>::value;  
template<class T> constexpr bool has_defer_member_v = has_defer_member<T>::value;  
template<class T> constexpr bool has_sync_execute_member_v = has_sync_execute_member<T>::value;  
template<class T> constexpr bool has_async_execute_member_v = has_async_execute_member<T>::value;  
template<class T> constexpr bool has_async_post_member_v = has_async_post_member<T>::value;  
template<class T> constexpr bool has_async_defer_member_v = has_async_defer_member<T>::value;  
template<class T> constexpr bool has_then_execute_member_v = has_then_execute_member<T>::value;  
template<class T> constexpr bool has_bulk_execute_member_v = has_bulk_execute_member<T>::value;  
template<class T> constexpr bool has_bulk_post_member_v = has_bulk_post_member<T>::value;  
template<class T> constexpr bool has_bulk_defer_member_v = has_bulk_defer_member<T>::value;  
template<class T> constexpr bool has_bulk_sync_execute_member_v = has_bulk_sync_execute_member<T>::value;  
template<class T> constexpr bool has_bulk_async_execute_member_v = has_bulk_async_execute_member<T>::value;  
template<class T> constexpr bool has_bulk_async_post_member_v = has_bulk_async_post_member<T>::value;  
template<class T> constexpr bool has_bulk_async_defer_member_v = has_bulk_async_defer_member<T>::value;  
template<class T> constexpr bool has_bulk_then_execute_member_v = has_bulk_then_execute_member<T>::value;
```

```
// Free function detection type traits:
```

```
template<class T> struct has_execute_free_function;  
template<class T> struct has_post_free_function;  
template<class T> struct has_defer_free_function;  
template<class T> struct has_sync_execute_free_function;  
template<class T> struct has_async_execute_free_function;  
template<class T> struct has_async_post_free_function;  
template<class T> struct has_async_defer_free_function;  
template<class T> struct has_then_execute_free_function;  
template<class T> struct has_bulk_execute_free_function;  
template<class T> struct has_bulk_post_free_function;  
template<class T> struct has_bulk_defer_free_function;  
template<class T> struct has_bulk_sync_execute_free_function;  
template<class T> struct has_bulk_async_execute_free_function;  
template<class T> struct has_bulk_async_post_free_function;  
template<class T> struct has_bulk_async_defer_free_function;  
template<class T> struct has_bulk_then_execute_free_function;
```

```
template<class T> constexpr bool has_execute_free_function_v = has_execute_free_function<T>::value;  
template<class T> constexpr bool has_post_free_function_v = has_post_free_function<T>::value;  
template<class T> constexpr bool has_defer_free_function_v = has_defer_free_function<T>::value;  
template<class T> constexpr bool has_sync_execute_free_function_v = has_sync_execute_free_function<T>::value;  
template<class T> constexpr bool has_async_execute_free_function_v = has_async_execute_free_function<T>::value;  
template<class T> constexpr bool has_async_post_free_function_v = has_async_post_free_function<T>::value;  
template<class T> constexpr bool has_async_defer_free_function_v = has_async_defer_free_function<T>::value;  
template<class T> constexpr bool has_then_execute_free_function_v = has_then_execute_free_function<T>::value;  
template<class T> constexpr bool has_bulk_execute_free_function_v = has_bulk_execute_free_function<T>::value;  
template<class T> constexpr bool has_bulk_post_free_function_v = has_bulk_post_free_function<T>::value;  
template<class T> constexpr bool has_bulk_defer_free_function_v = has_bulk_defer_free_function<T>::value;  
template<class T> constexpr bool has_bulk_sync_execute_free_function_v = has_bulk_sync_execute_free_function<T>::value;  
template<class T> constexpr bool has_bulk_async_execute_free_function_v = has_bulk_async_execute_free_function<T>::value;  
template<class T> constexpr bool has_bulk_async_post_free_function_v = has_bulk_async_post_free_function<T>::value;  
template<class T> constexpr bool has_bulk_async_defer_free_function_v = has_bulk_async_defer_free_function<T>::value;  
template<class T> constexpr bool has_bulk_then_execute_free_function_v = has_bulk_then_execute_free_function<T>::value;
```

```
// Customization points:
```

```
namespace {  
    constexpr unspecified execute = unspecified;  
    constexpr unspecified post = unspecified;  
    constexpr unspecified defer = unspecified;  
    constexpr unspecified sync_execute = unspecified;
```

```
constexpr unspecified async_execute = unspecified;
constexpr unspecified async_post = unspecified;
constexpr unspecified async_defer = unspecified;
constexpr unspecified then_execute = unspecified;
constexpr unspecified bulk_execute = unspecified;
constexpr unspecified bulk_post = unspecified;
constexpr unspecified bulk_defer = unspecified;
constexpr unspecified bulk_sync_execute = unspecified;
constexpr unspecified bulk_async_execute = unspecified;
constexpr unspecified bulk_async_post = unspecified;
constexpr unspecified bulk_async_defer = unspecified;
constexpr unspecified bulk_then_execute = unspecified;
}
```

```
// Customization point type traits:
```

```
template<class T> struct can_execute;
template<class T> struct can_post;
template<class T> struct can_defer;
template<class T> struct can_sync_execute;
template<class T> struct can_async_execute;
template<class T> struct can_async_post;
template<class T> struct can_async_defer;
template<class T> struct can_then_execute;
template<class T> struct can_bulk_execute;
template<class T> struct can_post_execute;
template<class T> struct can_defer_execute;
template<class T> struct can_bulk_sync_execute;
template<class T> struct can_bulk_async_execute;
template<class T> struct can_bulk_async_post;
template<class T> struct can_bulk_async_defer;
template<class T> struct can_bulk_then_execute;
```

```
template<class T> constexpr bool can_execute_v = can_execute<T>::value;
template<class T> constexpr bool can_post_v = can_post<T>::value;
template<class T> constexpr bool can_defer_v = can_defer<T>::value;
template<class T> constexpr bool can_sync_execute_v = can_sync_execute<T>::value;
template<class T> constexpr bool can_async_execute_v = can_async_execute<T>::value;
template<class T> constexpr bool can_async_post_v = can_async_post<T>::value;
template<class T> constexpr bool can_async_defer_v = can_async_defer<T>::value;
template<class T> constexpr bool can_then_execute_v = can_then_execute<T>::value;
template<class T> constexpr bool can_bulk_execute_v = can_bulk_execute<T>::value;
template<class T> constexpr bool can_bulk_post_v = can_bulk_post<T>::value;
template<class T> constexpr bool can_bulk_defer_v = can_bulk_defer<T>::value;
template<class T> constexpr bool can_bulk_sync_execute_v = can_bulk_sync_execute<T>::value;
template<class T> constexpr bool can_bulk_async_execute_v = can_bulk_async_execute<T>::value;
template<class T> constexpr bool can_bulk_async_post_v = can_bulk_async_post<T>::value;
template<class T> constexpr bool can_bulk_async_defer_v = can_bulk_async_defer<T>::value;
template<class T> constexpr bool can_bulk_then_execute_v = can_bulk_then_execute<T>::value;
```

```
// Executor type traits:
```

```
template<class T> struct is_one_way_executor;
template<class T> struct is_non_blocking_one_way_executor;
template<class T> struct is_two_way_executor;
template<class T> struct is_bulk_two_way_executor;
```

```
template<class T> constexpr bool is_one_way_executor_v = is_one_way_executor<T>::value;
template<class T> constexpr bool is_non_blocking_one_way_executor_v = is_non_blocking_one_way_executor<T>::value;
template<class T> constexpr bool is_two_way_executor_v = is_two_way_executor<T>::value;
template<class T> constexpr bool is_bulk_two_way_executor_v = is_bulk_two_way_executor<T>::value;
```

```
template<class Executor> struct executor_context;
```

```
template<class Executor>
    using executor_context_t = typename executor_context<Executor>::type;
```

```
template<class Executor, class T> struct executor_future;
```

```
template<class Executor, class T>
    using executor_future_t = typename executor_future<Executor, T>::type;
```

```
struct other_execution_mapping_tag {};
struct thread_execution_mapping_tag {};
struct unique_thread_execution_mapping_tag {};
```

```
template<class Executor> struct executor_execution_mapping_category;
```

```
template<class Executor>
```

```

using executor_execution_mapping_category_t = typename executor_execution_mapping_catetory<Executor>::type;

struct blocking_execution_tag {};
struct possibly_blocking_execution_tag {};
struct non_blocking_execution_tag {};

template<class Executor> struct executor_execute_blocking_category;

template<class Executor>
    using executor_execute_blocking_category_t = typename executor_execute_blocking_category<Executor>::type;

// Bulk executor traits:

struct sequenced_execution_tag {};
struct parallel_execution_tag {};
struct unsequenced_execution_tag {};

template<class Executor> struct executor_execution_category;

template<class Executor>
    using executor_execution_category_t = typename executor_execution_category<Executor>::type;

template<class Executor> struct executor_shape;

template<class Executor>
    using executor_shape_t = typename executor_shape<Executor>::type;

template<class Executor> struct executor_index;

template<class Executor>
    using executor_index_t = typename executor_index<Executor>::type;

// Executor work guard:

template <class Executor>
    class executor_work_guard;

// Polymorphic executor wrappers:

class one_way_executor;
class non_blocking_one_way_executor;
class two_way_executor;

} // namespace execution
} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std

```

2.1 Requirements

2.1.1 Customization point objects

(The following text has been adapted from the draft Ranges Technical Specification.)

A *customization point object* is a function object (C++ Std, [function.objects]) with a literal class type that interacts with user-defined types while enforcing semantic requirements on that interaction.

The type of a customization point object shall satisfy the requirements of CopyConstructible (C++Std [copyconstructible]) and Destructible (C++Std [destructible]).

All instances of a specific customization point object type shall be equal.

Let t be a (possibly const) customization point object of type τ , and $\text{args} \dots$ be a parameter pack expansion of some parameter pack $\text{Args} \dots$. The customization point object t shall be callable as $t(\text{args} \dots)$ when the types of $\text{Args} \dots$ meet the requirements specified in that customization point object's definition. Otherwise, τ shall not have a function call operator that participates in overload resolution.

Each customization point object type constrains its return type to satisfy some particular type requirements.

The library defines several named customization point objects. In every translation unit where such a name is defined, it shall refer to the same instance of the customization point object.

[*Note:* Many of the customization points objects in the library evaluate function call expressions with an unqualified name which results in a call to a user-defined function found by argument dependent name lookup (C++Std [basic.lookup.argdep]). To

preclude such an expression resulting in a call to unconstrained functions with the same name in namespace `std`, customization point objects specify that lookup for these expressions is performed in a context that includes deleted overloads matching the signatures of overloads defined in namespace `std`. When the deleted overloads are viable, user-defined overloads must be more specialized (C++Std [temp.func.order]) to be used by a customization point object. *--end note*

2.1.2 Future requirements

A type `F` meets the `Future` requirements for some value type `T` if `F` is `std::experimental::future<T>` (defined in the C++ Concurrency TS, ISO/IEC TS 19571:2016). *[Note: This concept is included as a placeholder to be elaborated, with the expectation that the elaborated requirements for `Future` will expand the applicability of some executor customization points. --end note]*

2.1.3 ProtoAllocator requirements

A type `A` meets the `ProtoAllocator` requirements if `A` is `CopyConstructible` (C++Std [copyconstructible]), `Destructible` (C++Std [destructible]), and `allocator_traits<A>::rebind_alloc<U>` meets the allocator requirements (C++Std [allocator.requirements]), where `U` is an object type. *[Note: For example, `std::allocator<void>` meets the proto-allocator requirements but not the allocator requirements. --end note]* No comparison operator, copy operation, move operation, or swap operation on these types shall exit via an exception.

2.1.4 ExecutionContext requirements

A type meets the `ExecutionContext` requirements if it satisfies the `EqualityComparable` requirements (C++Std [equalitycomparable]). No comparison operator on these types shall exit via an exception.

2.1.5 Requirements on execution functions

An execution function is a member function of the form:

```
x.e(...)
```

or a free function of the form:

```
e(x, ...)
```

where `x` denotes an executor object, `e` denotes the function name and `...` denotes the parameters.

Each execution function is made up from a combination of three properties: its **blocking semantics**, **directionality**, and **cardinality**. The combination of these properties determines the execution function's name, parameters, and semantics.

2.1.5.1 Naming of execution functions

The name of an execution function is determined by the combination of its properties. A word or prefix is associated with each property, and these are concatenated in the order below.

Cardinality	Directionality	Blocking semantics
"" or "bulk_"	"" or "sync_" or "async_" or "then_"	"execute" or "post" or "defer"

2.1.5.2 Semantics of execution functions

The parameters of the execution function and semantics that apply to the the execution function and to the execution agents created by it are determined by the combination of its properties. Parameters and semantics are added to an execution function for each of the properties. Whenever there is a conflict of semantics the presedence is resolved in the order below.

Cardinality > Directionality > Blocking semantics

2.1.5.3 Blocking semantics

The blocking semantics of an execution function may be one of the following:

- *Potentially blocking:* The execution function may block the caller pending completion of the submitted function objects. Execution functions having potentially blocking semantics are named `execute`.
- *Non-blocking:* The execution function shall not block the caller pending completion of the submitted function objects. Execution functions having non-blocking semantics are named `post` or `defer`.

2.1.5.3.1 Requirements on execution functions having potentially blocking semantics

In the Table below, `x` denotes a (possibly `const`) executor object of type `X` and `f` denotes a function object of type `F&&` callable as `DECAY_COPY(std::forward<F>(f))()` and where `decay_t<F>` satisfies the `MoveConstructible` requirements.

Expression	Return Type	Operational semantics
<code>x.execute(f, ...)</code> <code>execute(x, f, ...)</code>	<code>void</code>	Creates an execution agent with forward progress guarantees of <code>executor_execution_mapping_category_t<X></code> which invokes <code>DECAY_COPY(std::forward<F>(f))()</code> at most once, with the call to <code>DECAY_COPY</code> being evaluated in the thread that called <code>execute</code>. May block forward progress of the caller until <code>DECAY_COPY(std::forward<F>(f))()</code> finishes execution. The invocation of <code>execute</code> synchronizes with (C++Std [intro.multithread]) the invocation of <code>f</code>.

2.1.5.3.2 Requirements on execution functions having non-blocking semantics

In the Table below, `x` denotes a (possibly `const`) executor object of type `X` and `f` denotes a function object of type `F&&` callable as `DECAY_COPY(std::forward<F>(f))()` and where `decay_t<F>` satisfies the `MoveConstructible` requirements.

Expression	Return Type	Operational semantics
<code>x.post(f, ...)</code> <code>post(x, f, ...)</code>	<code>void</code>	Creates an execution agent with forward progress guarantees of <code>executor_execution_mapping_category_t<X></code> which invokes <code>DECAY_COPY(std::forward<F>(f))()</code> at most once, with the call to <code>DECAY_COPY</code> being evaluated in the thread that called <code>post</code>. Shall not block forward progress of the caller until <code>DECAY_COPY(std::forward<F>(f))()</code> finishes execution. The invocation of <code>execute</code> synchronizes with (C++Std [intro.multithread]) the invocation of <code>f</code>.
<code>x.defer(f, ...)</code> <code>defer(x, f, ...)</code>	<code>void</code>	Creates an execution agent with forward progress guarantees of <code>executor_execution_mapping_category_t<X></code> invokes <code>DECAY_COPY(std::forward<F>(f))()</code> at most once, with the call to <code>DECAY_COPY</code> being evaluated in the thread that called <code>defer</code>. Shall not block forward progress of the caller until <code>DECAY_COPY(std::forward<F>(f))()</code> finishes execution. The invocation of <code>execute</code> synchronizes with (C++Std [intro.multithread]) the invocation of <code>f</code>.

2.1.5.4 Directionality

The directionality property of an execution function may be one of the following:

- *One-way*: The execution function creates execution agents without a channel for awaiting the completion of a submitted function object or for obtaining its result. *Note*: That is, the executor provides fire-and-forget semantics. *--end note*] The names of execution functions having one-way directionality do not have an associated prefix.
- *Synchronous two-way*: The execution function blocks until execution of the submitted function is complete, and returns the result. The names of execution functions having synchronous two-way directionality have the prefix `sync_`.
- *Asynchronous two-way*: The execution function returns a `Future` for awaiting the completion of a submitted function object

and obtaining its result. The names of execution functions having asynchronous two-way directionality have the prefix `async_` or `then_`.

2.1.5.4.1 Requirements on execution functions of one-way directionality

In the Table below, `x` denotes a (possibly `const`) executor object of type `X`, `'e'` denotes an expression from the requirements on blocking semantics and `f` denotes a function object of type `F&&` callable as `DECAY_COPY(std::forward<F>(f))()` and where `decay_t<F>` satisfies the `MoveConstructible` requirements.

Expression	Return Type	Operational semantics
<code>x.'e'(...)</code> <code>'e'(x, ...)</code>	<code>void</code>	[<i>Note:</i> If <code>f()</code> exits via an exception, the behavior is specific to the executor. -- <i>end note.</i>]

2.1.5.4.2 Requirements on execution functions of synchronous two-way directionality

In the Table below, `x` denotes a (possibly `const`) executor object of type `X`, `'e'` denotes an expression from the requirements on blocking semantics and `f` denotes a function object of type `F&&` callable as `DECAY_COPY(std::forward<F>(f))()` and where `decay_t<F>` satisfies the `MoveConstructible` requirements.

Expression	Return Type	Operational semantics
		Returns the result of <code>f()</code> .
<code>x.sync_'e'(...)</code> <code>sync_'e'(x, ...)</code>	<code>R</code>	Throws any exception thrown by <code>f()</code> . Must block forward progress of the caller until <code>DECAY_COPY(std::forward<F>(f))()</code> finishes execution.

2.1.5.4.3 Requirements on execution functions of asynchronous two-way directionality

In the Table below, `x` denotes a (possibly `const`) executor object of type `X`, `'e'` denotes an expression from the requirements on blocking semantics, `f` denotes a function object of type `F&&` callable as `DECAY_COPY(std::forward<F>(f))()` and where `decay_t<F>` satisfies the `MoveConstructible` requirements and `pr` denotes a `Future` object whose result is `pr`.

Expression	Return Type	Operational semantics
<code>x.async_'e'(...)</code> <code>async_'e'(x, ...)</code>	A type that satisfies the <code>Future</code> requirements for the value type <code>R</code> .	Stores the result of <code>f()</code> , or any exception thrown by <code>f()</code> , in the associated shared state of the resulting <code>Future</code> .
<code>x.then_'e'(..., pr, ...)</code> <code>then_'e'(x, ..., pr, ...)</code>	A type that satisfies the <code>Future</code> requirements for the value type <code>R</code> .	Stores the result of <code>f(pr)</code> , or any exception thrown by <code>f(pr)</code> , in the associated shared state of the resulting <code>Future</code> .

2.1.5.5 Cardinality

The cardinality property of an execution function may be one of the following:

- *Single:* The execution function creates a single execution agent. The names of execution functions having single cardinality do not have an associated prefix.
- *Bulk:* The execution function creates multiple execution agents from a single invocation, with the number determined at runtime. The names of execution functions having bulk cardinality have the prefix `bulk_`.

2.1.5.5.1 Requirements on execution functions of single cardinality

In the Table below, `x` denotes a (possibly `const`) executor object of type `X`, `'e'` denotes an expression from the requirements on directionality, `'ret'` denotes the return type of the execution function from previous properties, `f` denotes a function object of type `F&&` callable as `DECAY_COPY(std::forward<F>(f))()` and where `decay_t<F>` satisfies the `MoveConstructible` requirements, and `a` denotes a (possibly `const`) value of type `A` satisfying the `ProtoAllocator` requirements.

Expression	Return Type	Operational semantics
		Executor implementations should use

```

x.'e'(...)
'e'(x, ...)      'ret'
x.'e'(..., a)
'e'(x, ..., a)

```

the supplied allocator (if any) to allocate any memory required to store the function object. Prior to invoking the function object, the executor shall deallocate any memory allocated. [Note: Executors defined in this Technical Specification always use the supplied allocator unless otherwise specified. *—end note*]

2.1.5.5.2 Requirements on execution functions of bulk cardinality

In the Table below,

- `x` denotes a (possibly const) executor object of type `X`,
- `'e'` denotes an expression from the requirements on directionality,
- `'ret'` denotes the return type of the execution function from previous properties,
- `f` denotes a function object of type `F&&` callable as `DECAY_COPY(std::forward<F>(f))(i, r, s)` and where `decay_t<F>` satisfies the `MoveConstructible` requirements where
 - `i` denotes an object whose type is `executor_index_t<X>`,
 - `r` denotes an object whose type is `R`,
 - `s` denotes an object whose type is `S`,
- `n` denotes a shape object whose type is `executor_shape_t<X>`,
- `rf` denotes a `CopyConstructible` function object with zero arguments whose result type is `R`,
- `sf` denotes a `CopyConstructible` function object with zero arguments whose result type is `S`,
- `pred` denotes a `Future` object whose result is `pr`.

Expression	Return Type	Operational semantics
		Creates a group of execution agents of shape <code>n</code> with forward progress guarantees of <code>executor_execution_mapping_category_t<X></code> which invokes <code>DECAY_COPY(std::forward<F>(f))(i, r, s)</code> if <code>'ret'</code> is non void otherwise invokes <code>DECAY_COPY(std::forward<F>(f))(i, s)</code> , with the call to <code>DECAY_COPY</code> being evaluated in the thread that called <code>bulk_'e'</code> .
<code>x.bulk_'e'(..., n, ..., [rf,] sf)</code> <code>bulk_'e'(x, ..., n, ..., [rf,] sf)</code>	<code>'ret'</code>	Parameter <code>rf</code> is only included in the execution function if <code>'ret'</code> is non void. The value of type <code>R</code> returned is the result of <code>rf()</code> if <code>'ret'</code> is non void. Invokes <code>rf()</code> on an unspecified execution agent. Invokes <code>sf()</code> on an unspecified execution agent.

2.1.5.6 Execution function combinations

The table below describes the execution member functions and non-member functions that can be supported by an executor category via various combinations of the execution function requirements.

Cardinality	Directionality	Blocking semantics	Member function	Free function
Single	One-way	Potentially blocking	<code>x.execute(f)</code> <code>x.execute(f, a)</code> <code>x.post(f)</code>	<code>execute(x, f)</code> <code>execute(x, f, a)</code> <code>post(x, f)</code>
Single	One-way	Non-blocking	<code>x.post(f, a)</code> <code>x.defer(f)</code> <code>x.defer(f, a)</code>	<code>post(x, f, a)</code> <code>defer(x, f)</code> <code>defer(x, f, a)</code>
Single	Two-way	Potentially blocking	<code>x.sync_execute(f)</code>	<code>sync_execute(x, f)</code>

	synchronous		<code>x.sync_execute(f, a)</code>	<code>sync_execute(x, f, a)</code>
Single	Two-way synchronous	Non-blocking	NA	NA
Single	Two-way asynchronous	Potentially blocking	<code>x.async_execute(f)</code> <code>x.async_execute(f, a)</code> <code>x.then_execute(f, pred)</code> <code>x.then_execute(f, pred, a)</code>	<code>async_execute(x, f)</code> <code>async_execute(x, f, a)</code> <code>then_execute(x, f, pred)</code> <code>then_execute(x, f, pred, a)</code>
Single	Two-way asynchronous	Non-blocking	<code>x.async_post(f)</code> <code>x.async_post(f, a)</code> <code>x.async_defer(f)</code> <code>x.async_defer(f, a)</code>	<code>async_post(x, f)</code> <code>x.async_post(x, f, a)</code> <code>async_defer(x, f)</code> <code>x.async_defer(x, f, a)</code>
Bulk	One-way	Potentially blocking	<code>x.bulk_execute(f, n, sf)</code>	<code>bulk_execute(x, f, n, sf)</code>
Bulk	One-way	Non-blocking	<code>x.bulk_post(f, n, sf)</code> <code>x.bulk_defer(f, n, sf)</code>	<code>bulk_post(x, f, n, sf)</code> <code>bulk_defer(x, f, n, sf)</code>
Bulk	Two-way synchronous	Potentially blocking	<code>x.bulk_sync_execute(f, n, rf, sf)</code>	<code>bulk_sync_execute(x, f, n, rf, sf)</code>
Bulk	Two-way synchronous	Non-blocking	NA	NA
Bulk	Two-way asynchronous	Potentially blocking	<code>x.bulk_async_execute(f, n, rf, sf)</code> <code>x.bulk_then_execute(f, n, pred, rf, sf)</code>	<code>bulk_async_execute(x, f, n, rf, sf)</code> <code>bulk_then_execute(x, f, n, pred, rf, sf)</code>
Bulk	Two-way asynchronous	Non-blocking	<code>x.bulk_async_post(f, n, rf, sf)</code> <code>x.bulk_async_defer(f, n, rf, sf)</code>	<code>bulk_async_post(x, f, n, rf, sf)</code> <code>bulk_async_defer(x, f, n, rf, sf)</code>

2.1.6 BaseExecutor requirements

A type `X` meets the `BaseExecutor` requirements if it satisfies the requirements of `CopyConstructible` (C++Std [copyconstructible]), `Destructible` (C++Std [destructible]), and `EqualityComparable` (C++Std [equalitycomparable]), as well as the additional requirements listed below.

No comparison operator, copy operation, move operation, swap operation, or member function context on these types shall exit via an exception.

The executor copy constructor, comparison operators, context member function, associated execution functions, and other member functions defined in refinements (TODO: what should this word be?) of the `BaseExecutor` requirements shall not introduce data races as a result of concurrent calls to those functions from different threads.

The destructor shall not block pending completion of the submitted function objects. [Note: The ability to wait for completion of submitted function objects may be provided by the associated execution context. *—end note*]

In the Table below, `x1` and `x2` denote (possibly const) values of type `X`, `mx1` denotes an `x`value of type `X`, and `u` denotes an identifier.

(Base executor requirements)		
Expression	Type	Assertion/note/pre-/post-condition
<code>X u(x1);</code>		Shall not exit via an exception. <i>Post:</i> <code>u == x1</code> and <code>u.context() == x1.context()</code> . Shall not exit via an exception.
<code>X u(mx1);</code>		<i>Post:</i> <code>u</code> equals the prior value of <code>mx1</code> and <code>u.context()</code> equals the prior value of <code>mx1.context()</code> . Returns <code>true</code> only if <code>x1</code> and <code>x2</code> can be interchanged with identical effects in any of the expressions defined in these type requirements (TODO and the other executor requirements defined in this Technical Specification). [Note: Returning <code>false</code> does not necessarily imply that the effects are not identical. <i>—end note</i>]
<code>x1 == x2</code>	<code>bool</code>	operator <code>==</code> shall be reflexive, symmetric, and transitive, and shall not exit via an exception.
<code>x1 != x2</code>	<code>bool</code>	Same as <code>!(x1 == x2)</code> .

<code>x1.context()</code>	E& OR const E& where E is a type that satisfies the <code>ExecutionContext</code> requirements.	Shall not exit via an exception. The comparison operators and member functions defined in these requirements (TODO and the other executor requirements defined in this Technical Specification) shall not alter the reference returned by this function.
---------------------------	---	--

2.1.7 `OneWayExecutor` requirements

The `OneWayExecutor` requirements specify requirements for executors which create execution agents without a channel for awaiting the completion of a submitted function object and obtaining its result. [Note: That is, the executor provides fire-and-forget semantics. --end note]

A type `X` satisfies the `OneWayExecutor` requirements if it satisfies the `BaseExecutor` requirements, and `can_execute_v<X>` is true.

2.1.8 `NonBlockingOneWayExecutor` requirements

The `NonBlockingOneWayExecutor` requirements refine the `OneWayExecutor` requirements by adding one-way operations that are guaranteed not to block the caller pending completion of submitted function objects.

A type `X` satisfies the `NonBlockingOneWayExecutor` requirements if it satisfies the `OneWayExecutor` requirements, `can_post_v<X>` is true, and `can_defer_v<X>` is true.

2.1.9 `TwoWayExecutor` requirements

The `TwoWayExecutor` requirements specify requirements for executors which creating execution agents with a channel for awaiting the completion of a submitted function object and obtaining its result.

A type `X` satisfies the `TwoWayExecutor` requirements if it satisfies the `BaseExecutor` requirements, `can_sync_execute_v<X>` is true, and `can_async_execute_v<X>` is true.

2.1.10 `BulkTwoWayExecutor` requirements

The `BulkTwoWayExecutor` requirements specify requirements for executors which create groups of execution agents in bulk from a single execution function with a channel for awaiting the completion of a submitted function object invoked by those execution agents and obtaining its result.

A type `X` satisfies the `BulkTwoWayExecutor` requirements if it satisfies the `BaseExecutor` requirements, `can_bulk_sync_execute_v<X>` is true, `can_bulk_async_execute_v<X>` is true, and `can_bulk_then_execute_v<X>` is true.

2.1.11 `ExecutorWorkTracker` requirements

The `ExecutorWorkTracker` requirements defines operations for tracking future work against an executor. These operations are used to advise an executor that function objects may be submitted to it at some point in the future.

A type `X` satisfies the `ExecutorWorkTracker` requirements if it satisfies the `BaseExecutor` requirements, as well as the additional requirements listed below.

No constructor, comparison operator, copy operation, move operation, swap operation, or member functions `on_work_started` and `on_work_finished` on these types shall exit via an exception.

The executor copy constructor, comparison operators, and other member functions defined in these requirements shall not introduce data races as a result of concurrent calls to those functions from different threads.

In the Table below, `x` denotes an object of type `X`,

(Executor Work Tracker requirements)		
Expression	Return Type	Assertion/note/pre-/post-condition
<code>x.on_work_started()</code>	<code>bool</code>	Shall not exit via an exception. Must be paired with a corresponding subsequent call to <code>on_work_finished</code> . Returns <code>false</code> if the executor will not execute any further functions submitted to it; otherwise returns <code>true</code> . A return value of <code>true</code> does not guarantee that the executor will execute any further functions submitted to it. Shall not exit via an exception.

`x.on_work_finished()`

Precondition: A corresponding preceding call to `on_work_started` that returned `true`.

2.1.12 Member detection type traits

```
template<class T> struct has_execute_member;
template<class T> struct has_post_member;
template<class T> struct has_defer_member;
template<class T> struct has_sync_execute_member;
template<class T> struct has_async_execute_member;
template<class T> struct has_async_post_member;
template<class T> struct has_async_defer_member;
template<class T> struct has_then_execute_member;
template<class T> struct has_bulk_execute_member;
template<class T> struct has_bulk_post_member;
template<class T> struct has_bulk_defer_member;
template<class T> struct has_bulk_sync_execute_member;
template<class T> struct has_bulk_async_execute_member;
template<class T> struct has_bulk_async_post_member;
template<class T> struct has_bulk_async_defer_member;
template<class T> struct has_bulk_then_execute_member;
```

This sub-clause contains templates that may be used to query the properties of a type at compile time. Each of these templates is a `UnaryTypeTrait` (C++Std [meta.rqmts]) with a `BaseCharacteristic` of `true_type` if the corresponding condition is true, otherwise `false_type`.

Template	Condition	Preconditions
<code>template<class T> struct has_execute_member</code>	τ has a member function named <code>execute</code> that satisfies the syntactic requirements of a one-way, potentially blocking execution function of single cardinality.	τ is a complete type.
<code>template<class T> struct has_post_member</code>	τ has a member function named <code>post</code> that satisfies the syntactic requirements of a one-way, non-blocking execution function of single cardinality.	τ is a complete type.
<code>template<class T> struct has_defer_member</code>	τ has a member function named <code>defer</code> that satisfies the syntactic requirements of a one-way, non-blocking execution function of single cardinality.	τ is a complete type.
<code>template<class T> struct has_sync_execute_member</code>	τ has a member function named <code>sync_execute</code> that satisfies the syntactic requirements of a synchronous two-way execution function of single cardinality.	τ is a complete type.
<code>template<class T> struct has_async_execute_member</code>	τ has a member function named <code>async_execute</code> that satisfies the syntactic requirements of an asynchronous two-way, potentially blocking execution function of single cardinality.	τ is a complete type.
<code>template<class T> struct has_async_post_member</code>	τ has a member function named <code>async_post</code> that satisfies the syntactic requirements of an asynchronous two-way, non-blocking execution function of single cardinality.	τ is a complete type.
<code>template<class T> struct has_async_defer_member</code>	τ has a member function named <code>async_defer</code> that satisfies the syntactic requirements of an asynchronous two-way, non-blocking execution function of single cardinality.	τ is a complete type.
	τ has a member function named	

<pre>template<class T> struct has_then_execute_member</pre>	<code>then_execute</code> that satisfies the syntactic requirements of an asynchronous two-way, potentially blocking execution function of single cardinality. τ has a member function named <code>bulk_execute</code> that satisfies the syntactic requirements of a one-way, potentially blocking execution function of bulk cardinality. τ has a member function named <code>bulk_post</code> that satisfies the syntactic requirements of a one-way, non-blocking execution function of bulk cardinality. τ has a member function named <code>bulk_defer</code> that satisfies the syntactic requirements of a one-way, non-blocking execution function of bulk cardinality. τ has a member function named <code>bulk_sync_execute</code> that satisfies the syntactic requirements of a synchronous two-way execution function of bulk cardinality. τ has a member function named <code>bulk_async_execute</code> that satisfies the syntactic requirements of an asynchronous two-way, potentially blocking execution function of bulk cardinality. τ has a member function named <code>bulk_async_post</code> that satisfies the syntactic requirements of an asynchronous two-way, non-blocking execution function of bulk cardinality. τ has a member function named <code>bulk_async_defer</code> that satisfies the syntactic requirements of an asynchronous two-way, non-blocking execution function of bulk cardinality. τ has a member function named <code>bulk_then_execute</code> that satisfies the syntactic requirements of an asynchronous two-way, potentially blocking execution function of bulk cardinality.	τ is a complete type. τ is a complete type. τ is a complete type. τ is a complete type. τ is a complete type. τ is a complete type. τ is a complete type. τ is a complete type.
<pre>template<class T> struct has_bulk_execute_member</pre>		
<pre>template<class T> struct has_bulk_post_member</pre>		
<pre>template<class T> struct has_bulk_defer_member</pre>		
<pre>template<class T> struct has_bulk_sync_execute_member</pre>		
<pre>template<class T> struct has_bulk_async_execute_member</pre>		
<pre>template<class T> struct has_bulk_async_post_member</pre>		
<pre>template<class T> struct has_bulk_async_defer_member</pre>		
<pre>template<class T> struct has_bulk_then_execute_member</pre>		

2.1.13 Free function detection type traits

```
template<class T> struct has_execute_free_function;
template<class T> struct has_post_free_function;
template<class T> struct has_defer_free_function;
template<class T> struct has_sync_execute_free_function;
template<class T> struct has_async_execute_free_function;
template<class T> struct has_async_post_free_function;
template<class T> struct has_async_defer_free_function;
template<class T> struct has_then_execute_free_function;
template<class T> struct has_bulk_execute_free_function;
template<class T> struct has_bulk_post_free_function;
template<class T> struct has_bulk_defer_free_function;
template<class T> struct has_bulk_sync_execute_free_function;
template<class T> struct has_bulk_async_execute_free_function;
```

```
template<class T> struct has_bulk_async_post_free_function;
template<class T> struct has_bulk_async_defer_free_function;
template<class T> struct has_bulk_then_execute_free_function;
```

This sub-clause contains templates that may be used to query the properties of a type at compile time. Each of these templates is a `UnaryTypeTrait` (C++Std [meta.rqmts]) with a `BaseCharacteristic` of `true_type` if the corresponding condition is true, otherwise `false_type`.

Template	Condition	Preconditions
<pre>template<class T> struct has_execute_free_function</pre>	There exists a free function named <code>execute</code> taking an executor of type τ that satisfies the syntactic requirements of a one-way, potentially blocking execution function of single cardinality.	τ is a complete type.
<pre>template<class T> struct has_post_free_function</pre>	There exists a free function named <code>post</code> taking an executor of type τ that satisfies the syntactic requirements of a one-way, non-blocking execution function of single cardinality.	τ is a complete type.
<pre>template<class T> struct has_defer_free_function</pre>	There exists a free function named <code>defer</code> taking an executor of type τ that satisfies the syntactic requirements of a one-way, non-blocking execution function of single cardinality.	τ is a complete type.
<pre>template<class T> struct has_sync_execute_free_function</pre>	There exists a free function named <code>sync_execute</code> taking an executor of type τ that satisfies the syntactic requirements of a synchronous two-way execution function of single cardinality.	τ is a complete type.
<pre>template<class T> struct has_async_execute_free_function</pre>	There exists a free function named <code>async_execute</code> taking an executor of type τ that satisfies the syntactic requirements of an asynchronous two-way, potentially blocking execution function of single cardinality.	τ is a complete type.
<pre>template<class T> struct has_async_post_free_function</pre>	There exists a free function named <code>async_post</code> taking an executor of type τ that satisfies the syntactic requirements of an asynchronous two-way, non-blocking execution function of single cardinality.	τ is a complete type.
<pre>template<class T> struct has_async_defer_free_function</pre>	There exists a free function named <code>async_defer</code> taking an executor of type τ that satisfies the syntactic requirements of an asynchronous two-way, non-blocking execution function of single cardinality.	τ is a complete type.
<pre>template<class T> struct has_then_execute_free_function</pre>	There exists a free function named <code>then_execute</code> taking an executor of type τ that satisfies the syntactic requirements of an asynchronous two-way, potentially blocking execution function of single cardinality.	τ is a complete type.
<pre>template<class T> struct has_bulk_execute_free_function</pre>	There exists a free function named <code>bulk_execute</code> taking an executor of type τ that satisfies the syntactic requirements of a one-way,	τ is a complete type.


```
template<class T>
struct has_bulk_post_free_function
```

potentially blocking execution
function of bulk cardinality.

There exists a free function named `bulk_post` taking an executor of type τ that satisfies the syntactic requirements of a one-way, non-blocking execution function of bulk cardinality. τ is a complete type.

```
template<class T>
struct has_bulk_defer_free_function
```

There exists a free function named `bulk_defer` taking an executor of type τ that satisfies the syntactic requirements of a one-way, non-blocking execution function of bulk cardinality. τ is a complete type.

```
template<class T>
struct has_bulk_sync_execute_free_function
```

There exists a free function named `bulk_sync_execute` taking an executor of type τ that satisfies the syntactic requirements of a synchronous two-way execution function of bulk cardinality. τ is a complete type.

```
template<class T>
struct has_bulk_async_execute_free_function
```

There exists a free function named `bulk_async_execute` taking an executor of type τ that satisfies the syntactic requirements of an asynchronous two-way, potentially blocking execution function of bulk cardinality. τ is a complete type.

```
template<class T>
struct has_bulk_async_post_free_function
```

There exists a free function named `bulk_async_post` taking an executor of type τ that satisfies the syntactic requirements of an asynchronous two-way, non-blocking execution function of bulk cardinality. τ is a complete type.

```
template<class T>
struct has_bulk_async_defer_free_function
```

There exists a free function named `bulk_async_defer` taking an executor of type τ that satisfies the syntactic requirements of an asynchronous two-way, non-blocking execution function of bulk cardinality. τ is a complete type.

```
template<class T>
struct has_bulk_then_execute_free_function
```

There exists a free function named `bulk_then_execute` taking an executor of type τ that satisfies the syntactic requirements of an asynchronous two-way, potentially blocking execution function of bulk cardinality. τ is a complete type.

2.2 Executor customization points

Executor customization points are execution functions which adapt an executor's free and member execution functions to create execution agents. Executor customization points enable uniform use of executors in generic contexts.

When an executor customization point named *NAME* invokes a free execution function of the same name, overload resolution is performed in a context that includes the declaration `void NAME(auto&... args) = delete;`, where `sizeof...(args)` is the arity of the free execution function. This context also does not include a declaration of the executor customization point.

[*Note:* This provision allows executor customization points to call the executor's free, non-member execution function of the same name without recursion. *--end note*]

Whenever `std::experimental::concurrency_v2::execution::NAME(ARGS)` is a valid expression, that expression satisfies the syntactic requirements for the free execution function named *NAME* with arity `sizeof...(ARGS)` with that free execution function's semantics.

2.2.1 execute

```
namespace {  
    constexpr unspecified execute = unspecified;  
}
```

The name `execute` denotes a customization point. The effect of the expression

`std::experimental::concurrency_v2::execution::execute(E, F, A...)` for some expressions `E` and `F`, and where `A...` represents 0 or 1 expressions, is equivalent to:

- `(E).execute(F, A...)` if `has_execute_member_v<decay_t<decltype(E)>>` is true.
- Otherwise, `execute(E, F, A...)` if `has_execute_free_function_v<decay_t<decltype(E)>>` is true.
- Otherwise, `std::experimental::concurrency_v2::execution::execute(E, F, A...)` is ill-formed.

2.2.2 post

```
namespace {  
    constexpr unspecified post = unspecified;  
}
```

The name `post` denotes a customization point. The effect of the expression `std::experimental::concurrency_v2::execution::post(E, F, A...)` for some expressions `E` and `F`, and where `A...` represents 0 or 1 expressions, is equivalent to:

- `(E).post(F, A...)` if `has_post_member_v<decay_t<decltype(E)>>` is true.
- Otherwise, `post(E, F, A...)` if `has_post_free_function_v<decay_t<decltype(E)>>` is true.
- Otherwise, `std::experimental::concurrency_v2::execution::execute(E, F, A...)` if `can_execute_v<decay_t<decltype(E)>> && is_same_v<execution_execute_blocking_category_t<decay_t<decltype(E)>>, non_blocking_execution_tag>` is true.
- Otherwise, `std::experimental::concurrency_v2::execution::post(E, F, A...)` is ill-formed.

2.2.3 defer

```
namespace {  
    constexpr unspecified defer = unspecified;  
}
```

The name `defer` denotes a customization point. The effect of the expression

`std::experimental::concurrency_v2::execution::defer(E, F, A...)` for some expressions `E` and `F`, and where `A...` represents 0 or 1 expressions, is equivalent to:

- `(E).defer(F, A...)` if `has_defer_member_v<decay_t<decltype(E)>>` is true.
- Otherwise, `defer(E, F, A...)` if `has_defer_free_function_v<decay_t<decltype(E)>>` is true.
- Otherwise, `std::experimental::concurrency_v2::execution::execute(E, F, A...)` if `can_execute_v<decay_t<decltype(E)>> && is_same_v<execution_execute_blocking_category_t<decay_t<decltype(E)>>, non_blocking_execution_tag>` is true.
- Otherwise, `std::experimental::concurrency_v2::execution::defer(E, F, A...)` is ill-formed.

2.2.4 sync_execute

```
namespace {  
    constexpr unspecified sync_execute = unspecified;  
}
```

The name `sync_execute` denotes a customization point. The effect of the expression

`std::experimental::concurrency_v2::execution::sync_execute(E, F, A...)` for some expressions `E` and `F`, and where `A...` represents 0 or 1 expressions, is equivalent to:

- `(E).sync_execute(F, A...)` if `has_sync_execute_member_v<decay_t<decltype(E)>>` is true.
- Otherwise, `sync_execute(E, F, A...)` if `has_sync_execute_free_function_v<decay_t<decltype(E)>>` is true.
- Otherwise, `std::experimental::concurrency_v2::execution::async_execute(E, F, A...).get()` if `can_async_execute_v<decay_t<decltype(E)>>` is true.

- Otherwise, `std::experimental::concurrency_v2::execution::sync_execute(E, F, A...)` is ill-formed.

2.2.5 `async_execute`

```
namespace {
    constexpr unspecified async_execute = unspecified;
}
```

The name `async_execute` denotes a customization point. The effect of the expression

`std::experimental::concurrency_v2::execution::async_execute(E, F, A...)` for some expressions `E` and `F`, and where `A...` represents 0 or 1 expressions, is equivalent to:

- `(E).async_execute(F, A...)` if `has_async_execute_member_v<decay_t<decltype(E)>>` is true.
- Otherwise, `async_execute(E, F, A...)` if `has_async_execute_free_function_v<decay_t<decltype(E)>>` is true.
- Otherwise, if `can_execute_v<decay_t<decltype(E)>>` is true, creates an asynchronous provider with an associated shared state (C++Std [futures.state]). Calls `std::experimental::concurrency_v2::execution::execute(E, g, A...)` where `g` is a function object of unspecified type that performs `DECAY_COPY(F)()`, with the call to `DECAY_COPY` being performed in the thread that called `async_execute`. On successful completion of `DECAY_COPY(F)()`, the return value of `DECAY_COPY(F)()` is atomically stored in the shared state and the shared state is made ready. If `DECAY_COPY(F)()` exits via an exception, the exception is atomically stored in the shared state and the shared state is made ready. The result of the expression `std::experimental::concurrency_v2::execution::async_execute(E, F, A...)` is an object of type `std::future<result_of_t<decay_t<decltype(F)>>()>` that refers to the shared state.
- Otherwise, `std::experimental::concurrency_v2::execution::async_execute(E, F, A...)` is ill-formed.

2.2.6 `async_post`

```
namespace {
    constexpr unspecified async_post = unspecified;
}
```

The name `async_post` denotes a customization point. The effect of the expression

`std::experimental::concurrency_v2::execution::async_post(E, F, A...)` for some expressions `E` and `F`, and where `A...` represents 0 or 1 expressions, is equivalent to:

- `(E).async_post(F, A...)` if `has_async_post_member_v<decay_t<decltype(E)>>` is true.
- Otherwise, `async_post(E, F, A...)` if `has_async_post_free_function_v<decay_t<decltype(E)>>` is true.
- Otherwise, `std::experimental::concurrency_v2::execution::async_execute(E, F, A...)` if `can_async_execute_v<decay_t<decltype(E)>> && is_same_v<execution_execute_blocking_category_t<decay_t<decltype(E)>>, non_blocking_execution_tag>` is true.
- Otherwise, if `can_post_v<decay_t<decltype(E)>>` is true, creates an asynchronous provider with an associated shared state (C++Std [futures.state]). Calls `std::experimental::concurrency_v2::execution::execute(E, g, A...)` where `g` is a function object of unspecified type that performs `DECAY_COPY(F)()`, with the call to `DECAY_COPY` being performed in the thread that called `async_post`. On successful completion of `DECAY_COPY(F)()`, the return value of `DECAY_COPY(F)()` is atomically stored in the shared state and the shared state is made ready. If `DECAY_COPY(F)()` exits via an exception, the exception is atomically stored in the shared state and the shared state is made ready. The result of the expression `std::experimental::concurrency_v2::execution::async_post(E, F, A...)` is an object of type `std::future<result_of_t<decay_t<decltype(F)>>()>` that refers to the shared state.
- Otherwise, `std::experimental::concurrency_v2::execution::async_post(E, F, A...)` is ill-formed.

2.2.7 `async_defer`

```
namespace {
    constexpr unspecified async_defer = unspecified;
}
```

The name `async_defer` denotes a customization point. The effect of the expression

`std::experimental::concurrency_v2::execution::async_defer(E, F, A...)` for some expressions `E` and `F`, and where `A...` represents 0 or 1 expressions, is equivalent to:

- `(E).async_defer(F, A...)` if `has_async_defer_member_v<decay_t<decltype(E)>>` is true.
- Otherwise, `async_defer(E, F, A...)` if `has_async_defer_free_function_v<decay_t<decltype(E)>>` is true.

- Otherwise, `std::experimental::concurrency_v2::execution::async_execute(E, F, A...)` if `can_async_execute_v<decay_t<decltype(E)>> && is_same_v<execution_execute_blocking_category_t<decay_t<decltype(E)>>, non_blocking_execution_tag> is true`.
- Otherwise, if `can_defer_v<decay_t<decltype(E)>> is true`, creates an asynchronous provider with an associated shared state (C++Std [futures.state]). Calls `std::experimental::concurrency_v2::execution::execute(E, g, A...)` where `g` is a function object of unspecified type that performs `DECAY_COPY(F)()`, with the call to `DECAY_COPY(F)()` being performed in the thread that called `async_defer`. On successful completion of `DECAY_COPY(F)()`, the return value of `DECAY_COPY(F)()` is atomically stored in the shared state and the shared state is made ready. If `DECAY_COPY(F)()` exits via an exception, the exception is atomically stored in the shared state and the shared state is made ready. The result of the expression `std::experimental::concurrency_v2::execution::async_defer(E, F, A...)` is an object of type `std::future<result_of_t<decay_t<decltype(F)>>()>` that refers to the shared state.
- Otherwise, `std::experimental::concurrency_v2::execution::async_defer(E, F, A...)` is ill-formed.

2.2.8 then_execute

```
namespace {
    constexpr unspecified then_execute = unspecified;
}
```

The name `then_execute` denotes a customization point. The effect of the expression

`std::experimental::concurrency_v2::execution::then_execute(E, F, P, A...)` for some expressions `E`, `F`, and `P`, and where `A...` represents 0 or 1 expressions, is equivalent to:

- `(E).then_execute(F, P, A...)` if `has_then_execute_member_v<decay_t<decltype(E)>> is true`.
- Otherwise, `then_execute(E, F, P, A...)` if `has_then_execute_free_function_v<decay_t<decltype(E)>> is true`.
- Otherwise, equivalent to

```
auto __g = [&f = forward<decltype(F)>(F)](decltype(P)& __predecessor_future)
{
    auto __predecessor_result = __predecessor_future.get();
    return __f(__predecessor_result);
}

return (P).then(E, std::move(__g));
```

when `P` is a non-void future. Otherwise,

```
auto __g = [&f = forward<decltype(F)>(F)](decltype(P)&)
{
    return __f();
}

return (P).then(E, std::move(__g));
```

- Otherwise, `std::experimental::concurrency_v2::execution::then_execute(E, F, P, A...)` is ill-formed

2.2.9 bulk_execute

```
namespace {
    constexpr unspecified bulk_execute = unspecified;
}
```

The name `bulk_execute` denotes a customization point. The effect of the expression

`std::experimental::concurrency_v2::execution::bulk_execute(E, F, S, SF)` for some expressions `E`, `F`, `S`, and `SF` is equivalent to:

- `(E).bulk_execute(F, S, SF)` if `has_bulk_execute_member_v<decay_t<decltype(E)>> is true`.
- Otherwise, `bulk_execute(E, F, S, SF)` if `has_bulk_execute_free_function_v<decay_t<decltype(E)>> is true`.
- Otherwise, `std::experimental::concurrency_v2::execution::bulk_execute(E, F, S, SF)` is ill-formed.

2.2.10 bulk_post

```
namespace {
    constexpr unspecified bulk_post = unspecified;
}
```

The name `bulk_post` denotes a customization point. The effect of the expression

`std::experimental::concurrency_v2::execution::bulk_post(E, F, S, SF)` for some expressions `E`, `F`, `S`, and `SF` is equivalent to:

- `(E).bulk_post(F, S, SF)` if `has_bulk_post_member_v<decay_t<decltype(E)>>` is true.
- Otherwise, `bulk_post(E, F, S, SF)` if `has_bulk_post_free_function_v<decay_t<decltype(E)>>` is true.
- Otherwise, `std::experimental::concurrency_v2::execution::bulk_execute(E, F, S, SF)` if `can_bulk_execute_v<decay_t<decltype(E)>> && is_same_v<execution_execute_blocking_category_t<decay_t<decltype(E)>>, non_blocking_execution_tag>` is true.
- Otherwise, `std::experimental::concurrency_v2::execution::bulk_post(E, F, S, SF)` is ill-formed.

2.2.11 bulk_defer

```
namespace {  
    constexpr unspecified bulk_defer = unspecified;  
}
```

The name `bulk_defer` denotes a customization point. The effect of the expression

`std::experimental::concurrency_v2::execution::bulk_defer(E, F, S, SF)` for some expressions `E`, `F`, `S`, and `SF` is equivalent to:

- `(E).bulk_defer(F, S, SF)` if `has_bulk_defer_member_v<decay_t<decltype(E)>>` is true.
- Otherwise, `bulk_defer(E, F, S, SF)` if `has_bulk_defer_free_function_v<decay_t<decltype(E)>>` is true.
- Otherwise, `std::experimental::concurrency_v2::execution::bulk_execute(E, F, S, SF)` if `can_bulk_execute_v<decay_t<decltype(E)>> && is_same_v<execution_execute_blocking_category_t<decay_t<decltype(E)>>, non_blocking_execution_tag>` is true.
- Otherwise, `std::experimental::concurrency_v2::execution::bulk_defer(E, F, S, SF)` is ill-formed.

2.2.12 bulk_sync_execute

```
namespace {  
    constexpr unspecified bulk_sync_execute = unspecified;  
}
```

The name `bulk_sync_execute` denotes a customization point. The effect of the expression

`std::experimental::concurrency_v2::execution::bulk_sync_execute(E, F, S, RF, SF)` for some expressions `E`, `F`, `S`, `RF`, and `SF` is equivalent to:

- `(E).bulk_sync_execute(F, S, RF, SF)` if `has_bulk_sync_execute_member_v<decay_t<decltype(E)>>` is true.
- Otherwise, `bulk_sync_execute(E, F, S, RF, SF)` if `has_bulk_sync_execute_free_function_v<decay_t<decltype(E)>>` is true.
- Otherwise, `std::experimental::concurrency_v2::execution::bulk_async_execute(E, F, S, RF, SF).get()` if `can_bulk_async_execute_v<decay_t<decltype(E)>>` is true.
- Otherwise, `std::experimental::concurrency_v2::execution::bulk_sync_execute(E, F, S, RF, SF)` is ill-formed.

2.2.13 bulk_async_execute

```
namespace {  
    constexpr unspecified bulk_async_execute = unspecified;  
}
```

The name `bulk_async_execute` denotes a customization point. The effect of the expression

`std::experimental::concurrency_v2::execution::bulk_async_execute(E, F, S, RF, SF)` for some expressions `E`, `F`, `S`, `RF`, and `SF` is equivalent to:

- `(E).bulk_async_execute(F, S, RF, SF)` if `has_bulk_async_execute_member_v<decay_t<decltype(E)>>` is true.
- Otherwise, `bulk_async_execute(E, F, S, RF, SF)` if `has_bulk_async_execute_free_function_v<decay_t<decltype(E)>>` is true.
- Otherwise, `std::experimental::concurrency_v2::execution::bulk_then_execute(E, F, S, std::experimental::make_ready_future(), RF, SF)` if `can_bulk_then_execute_v<decay_t<decltype(E)>>` is true.
- Otherwise, `std::experimental::concurrency_v2::execution::bulk_async_execute(E, F, S, RF, SF)` is ill-formed.

2.2.14 bulk_async_post

```
namespace {
    constexpr unspecified bulk_async_post = unspecified;
}
```

The name `bulk_async_post` denotes a customization point. The effect of the expression

`std::experimental::concurrency_v2::execution::bulk_async_post(E, F, S, RF, SF)` for some expressions `E`, `F`, `S`, `RF`, and `SF` is equivalent to:

- `(E).bulk_async_post(F, S, RF, SF)` if `has_bulk_async_post_member_v<decay_t<decltype(E)>>` is true.
- Otherwise, `bulk_async_post(E, F, S, RF, SF)` if `has_bulk_async_post_free_function_v<decay_t<decltype(E)>>` is true.
- Otherwise, `std::experimental::concurrency_v2::execution::bulk_async_execute(E, F, S, RF, SF)` if `can_bulk_async_execute_v<decay_t<decltype(E)>> && is_same_v<execution_execute_blocking_category_t<decay_t<decltype(E)>>, non_blocking_execution_tag>` is true.
- Otherwise, `std::experimental::concurrency_v2::execution::bulk_async_post(E, F)` is ill-formed.

2.2.15 `bulk_async_defer`

```
namespace {
    constexpr unspecified bulk_async_defer = unspecified;
}
```

The name `bulk_async_defer` denotes a customization point. The effect of the expression

`std::experimental::concurrency_v2::execution::bulk_async_defer(E, F, S, RF, SF)` for some expressions `E`, `F`, `S`, `RF`, and `SF` is equivalent to:

- `(E).bulk_async_defer(F, S, RF, SF)` if `has_bulk_async_defer_member_v<decay_t<decltype(E)>>` is true.
- Otherwise, `bulk_async_defer(E, F, S, RF, SF)` if `has_bulk_async_defer_free_function_v<decay_t<decltype(E)>>` is true.
- Otherwise, `std::experimental::concurrency_v2::execution::bulk_async_execute(E, F, S, RF, SF)` if `can_bulk_async_execute_v<decay_t<decltype(E)>> && is_same_v<execution_execute_blocking_category_t<decay_t<decltype(E)>>, non_blocking_execution_tag>` is true.
- Otherwise, `std::experimental::concurrency_v2::execution::bulk_async_defer(E, F)` is ill-formed.

2.2.16 `bulk_then_execute`

```
namespace {
    constexpr unspecified bulk_then_execute = unspecified;
}
```

The name `bulk_then_execute` denotes a customization point. The effect of the expression

`std::experimental::concurrency_v2::execution::bulk_then_execute(E, F, S, P, RF, SF)` for some expressions `E`, `F`, `S`, `P`, `RF`, and `SF` is equivalent to:

- `(E).bulk_then_execute(F, S, P, RF, SF)` if `has_bulk_then_execute_member_v<decay_t<decltype(E)>>` is true.
- Otherwise, `bulk_then_execute(E, F, S, P, RF, SF)` if `has_bulk_then_execute_free_function_v<decay_t<decltype(E)>>` is true.
- Otherwise, let `DE` be `decay_t<decltype(E)>`. If `can_then_execute_v<DE> && (has_bulk_sync_execute_member_v<DE> || has_bulk_sync_execute_free_function_v<DE> || has_bulk_async_execute_member_v<DE> || has_bulk_async_execute_free_function_v<DE>)` is true, equivalent to the following:

```
auto __f = F;

auto __g = [=](auto& __predecessor)
{
    return std::experimental::concurrency_v2::bulk_sync_execute(E, S, RF, SF,
        [=,&__predecessor](auto& __result, auto& __shared)
        {
            __f(__i, __predecessor, __result, __shared);
        });
};

return std::experimental::concurrency_v2::execution::then_execute(E, __g, P);
```

if `P` is a non-void future. Otherwise,

```
auto __f = F;
```

```

auto __g = [=]
{
    return std::experimental::concurrency_v2::bulk_sync_execute(E, S, RF, SF,
        [=](auto& __result, auto& __shared)
        {
            __f(__i, __result, __shared);
        });
};

return std::experimental::concurrency_v2::execution::then_execute(E, __g, P);

```

[*Note:* The explicit use of execution function detectors for `bulk_sync_execute` and `bulk_async_execute` above is intentional to avoid cycles in this code. --*end note*]

- Otherwise, `std::experimental::concurrency_v2::execution::bulk_then_execute(E, F, S, P, RF, SF)` is ill-formed.

2.2.17 Customization point type traits

```

template<class T> struct can_execute;
template<class T> struct can_post;
template<class T> struct can_defer;
template<class T> struct can_sync_execute;
template<class T> struct can_async_execute;
template<class T> struct can_async_post;
template<class T> struct can_async_defer;
template<class T> struct can_then_execute;
template<class T> struct can_bulk_execute;
template<class T> struct can_bulk_post;
template<class T> struct can_bulk_defer;
template<class T> struct can_bulk_sync_execute;
template<class T> struct can_bulk_async_execute;
template<class T> struct can_bulk_async_post;
template<class T> struct can_bulk_async_defer;
template<class T> struct can_bulk_then_execute;

```

This sub-clause contains templates that may be used to query the properties of a type at compile time. Each of these templates is a `UnaryTypeTrait` (C++Std [meta.rqmts]) with a `BaseCharacteristic` of `true_type` if the corresponding condition is true, otherwise `false_type`.

In the Table below,

- `t` denotes a (possibly const) executor object of type `T`,
- `f` denotes a function object of type `F&&` callable as `DECAY_COPY(std::forward<F>(f))()`, where `decay_t<F>` satisfies the `MoveConstructible` requirements.
- `bof` denotes a function object of type `F&&` callable as `DECAY_COPY(std::forward<F>(bof))(i, s)`,
 - where `i` denotes an object whose type is `executor_index_t<X>`,
 - where `s` denotes an object whose type is `S` and
 - where `decay_t<F>` satisfies the `CopyConstructible` requirements,
- `btf` denotes a function object of type `F&&` callable as `DECAY_COPY(std::forward<F>(btf))(i, r, s)`,
 - where `i` denotes an object whose type is `executor_index_t<X>`,
 - where `r` denotes an object whose type is `R`,
 - where `s` denotes an object whose type is `S` and
 - where `decay_t<F>` satisfies the `CopyConstructible` requirements,
- `bcf` denotes a function object of type `F&&` callable as `DECAY_COPY(std::forward<F>(bcf))(i, p, r, s)`,
 - where `i` denotes an object whose type is `executor_index_t<X>`,
 - where `p` denotes an object whose type is `P`,
 - where `r` denotes an object whose type is `R`,
 - where `s` denotes an object whose type is `S` and
 - where `decay_t<F>` satisfies the `CopyConstructible` requirements,
- `rf` denotes a `CopyConstructible` function object with zero arguments whose result type is `R`,
- `sf` denotes a `CopyConstructible` function object with zero arguments whose result type is `S`,
- `pred` denotes a `Future` object whose result type is `P` and
- `a` denotes a (possibly const) value of type `A` satisfying the `ProtoAllocator` requirements.

Template	Conditions	Preconditions
	The expressions	
<pre> template<class T> struct can_execute </pre>	<pre> std::experimental::concurrency_v2::execution::execute(t, f) and std::experimental::concurrency_v2::execution::execute(t, f, a) are well-formed. </pre>	<code>T</code> is a complete type.

template<class T> struct can_post	The expressions std::experimental::concurrency_v2::execution::post(t, f) and std::experimental::concurrency_v2::execution::post(t, f, a) are well-formed.	τ is a complete type.
template<class T> struct can_defer	The expressions std::experimental::concurrency_v2::execution::defer(t, f) and std::experimental::concurrency_v2::execution::defer(t, f, a) are well-formed.	τ is a complete type.
template<class T> struct can_sync_execute	The expressions std::experimental::concurrency_v2::execution::sync_execute(t, f) and std::experimental::concurrency_v2::execution::sync_execute(t, f, a) are well-formed.	τ is a complete type.
template<class T> struct can_async_execute	The expressions std::experimental::concurrency_v2::execution::async_execute(t, f) and std::experimental::concurrency_v2::execution::async_execute(t, f, a) are well-formed.	τ is a complete type.
template<class T> struct can_async_post	The expressions std::experimental::concurrency_v2::execution::async_post(t, f) and std::experimental::concurrency_v2::execution::async_post(t, f, a) are well-formed.	τ is a complete type.
template<class T> struct can_async_defer	The expressions std::experimental::concurrency_v2::execution::async_defer(t, f) and std::experimental::concurrency_v2::execution::async_defer(t, f, a) is well-formed.	τ is a complete type.
template<class T> struct can_then_execute	The expressions std::experimental::concurrency_v2::execution::then_execute(t, f, pred) and std::experimental::concurrency_v2::execution::then_execute(t, f, pred) are well-formed.	τ is a complete type.
template<class T> struct can_bulk_execute	The expression std::experimental::concurrency_v2::execution::bulk_execute(t, bof, s, sf) is well-formed.	τ is a complete type.
template<class T> struct can_bulk_post	The expression std::experimental::concurrency_v2::execution::bulk_post(t, bof, s, sf) is well-formed.	τ is a complete type.
template<class T> struct can_bulk_defer	The expression std::experimental::concurrency_v2::execution::bulk_defer(t, bof, s, sf) is well-formed.	τ is a complete type.
template<class T> struct can_bulk_sync_execute	The expression std::experimental::concurrency_v2::execution::bulk_sync_execute(t, btf, s, rf, sf) is well-formed.	τ is a complete type.
template<class T> struct can_bulk_async_execute	The expression std::experimental::concurrency_v2::execution::bulk_async_execute(t, btf, s, rf, sf) is well-formed.	τ is a complete type.
template<class T> struct can_bulk_async_post	The expression std::experimental::concurrency_v2::execution::bulk_async_post(t, btf, s, rf, sf) is well-formed.	τ is a complete type.
template<class T> struct can_bulk_async_defer	The expression std::experimental::concurrency_v2::execution::bulk_async_defer(t, btf, s, rf, sf) is well-formed.	τ is a complete type.
template<class T> struct can_bulk_then_execute	The expression std::experimental::concurrency_v2::execution::bulk_then_execute(t, bcf, s, pred, rf, sf) is well-formed.	τ is a complete type.

2.3 Executor type traits

2.3.1 Determining that a type satisfies executor category requirements

```
template<class T> struct is_one_way_executor;
template<class T> struct is_non_blocking_one_way_executor;
template<class T> struct is_two_way_executor;
template<class T> struct is_bulk_two_way_executor;
```


This sub-clause contains templates that may be used to query the properties of a type at compile time. Each of these templates is a `UnaryTypeTrait` (C++Std [meta.rqmts]) with a `BaseCharacteristic` of `true_type` if the corresponding condition is true, otherwise `false_type`.

Template	Condition	Preconditions
<pre>template<class T> struct is_one_way_executor</pre>	<code>T</code> meets the syntactic requirements for <code>OneWayExecutor</code> .	<code>T</code> is a complete type.
<pre>template<class T> struct is_non_blocking_one_way_executor</pre>	<code>T</code> meets the syntactic requirements for <code>NonBlockingOneWayExecutor</code> .	<code>T</code> is a complete type.
<pre>template<class T> struct is_two_way_executor</pre>	<code>T</code> meets the syntactic requirements for <code>TwoWayExecutor</code> .	<code>T</code> is a complete type.
<pre>template<class T> struct is_bulk_two_way_executor</pre>	<code>T</code> meets the syntactic requirements for <code>BulkTwoWayExecutor</code> .	<code>T</code> is a complete type.

2.3.2 Associated execution context type

```
template<class Executor>
struct executor_context
{
    using type = std::decay_t<decltype(declval<const Executor&>().context())>; // TODO check this
};
```

2.3.3 Associated future type

```
template<class Executor, class T>
struct executor_future
{
    using type = see below;
};
```

The type of `executor_future<Executor, T>::type` is determined as follows:

- if `is_two_way_executor<Executor>` is true, `decltype(declval<const Executor&>().async_execute( declval<T(*)()>()))`;
- otherwise, if `is_one_way_executor<Executor>` is true, `std::future<T>`;
- otherwise, the program is ill formed.

[*Note:* The effect of this specification is that all execute functions of an executor that satisfies the `TwoWayExecutor`, `NonBlockingTwoWayExecutor`, Or `BulkTwoWayExecutor` requirements must utilize the same future type, and that this future type is determined by `async_execute`. Programs may specialize this trait for user-defined `Executor` types. --end note]

2.3.4 Classifying the mapping of execution agents

```
struct other_execution_mapping_tag {};
struct thread_execution_mapping_tag {};
struct unique_thread_execution_mapping_tag {};

template<class Executor>
struct executor_execution_mapping_category
{
private:
    // exposition only
    template<class T>
    using helper = typename T::execution_mapping_category;

public:
    using type = std::experimental::detected_or_t<
        thread_execution_mapping_tag, helper, Executor
    >;
};
```

Components which create execution agents may use *execution mapping categories* to communicate the mapping of execution agents onto threads of execution. Execution mapping categories encode the characterisitics of that mapping, if it exists.

`other_execution_mapping_tag` indicates that execution agents created by a component may be mapped onto execution resources other than threads of execution.

`thread_execution_mapping_tag` indicates that execution agents created by a component are mapped onto threads of execution.

`unique_thread_execution_mapping_tag` indicates that each execution agent created by a component is mapped onto a new thread of execution.

[*Note*: A mapping of an execution agent onto a thread of execution implies the agent executes as-if on a `std::thread`. Therefore, the facilities provided by `std::thread`, such as thread-local storage, are available. `unique_thread_execution_mapping_tag` provides stronger guarantees, in particular that thread-local storage will not be shared between execution agents. --*end note*]

2.3.5 Classifying the blocking behavior of potentially blocking operations

```
struct blocking_execution_tag {};  
struct possibly_blocking_execution_tag {};  
struct non_blocking_execution_tag {};  
  
template<class Executor>  
struct executor_execute_blocking_category  
{  
    private:  
        // exposition only  
        template<class T>  
        using helper = typename T::blocking_category;  
  
    public:  
        using type = std::experimental::detected_or_t<  
            possibly_blocking_execution_tag, helper, Executor  
        >;  
};
```

Components which create possibly blocking execution may use *blocking categories* to communicate the way in which this execution blocks the progress of its caller.

`blocking_execution_tag` indicates that a component blocks its caller's progress pending the completion of the execution agents created by that component.

`possibly_blocking_execution_tag` indicates that a component may block its caller's progress pending the completion of the execution agents created by that component.

`non_blocking_execution_tag` indicates that a component does not block its caller's progress pending the completion of the execution agents created by that component.

Programs may use `executor_execute_blocking_category` to query the blocking behavior of executor customization points whose semantics allow the possibility of blocking.

[*Note*: These customization points which allow the possibility of blocking are `execute`, `async_execute`, `then_execute`, `bulk_execute`, `bulk_async_execute`, and `bulk_then_execute`. --*end note*]

2.4 Bulk executor traits

2.4.1 Classifying forward progress guarantees of groups of execution agents

```
struct sequenced_execution_tag {};  
struct parallel_execution_tag {};  
struct unsequenced_execution_tag {};  
  
template<class Executor>  
struct executor_execution_category  
{  
    private:  
        // exposition only  
        template<class T>  
        using helper = typename T::execution_category;  
  
    public:  
        using type = std::experimental::detected_or_t<  
            unsequenced_execution_tag, helper, Executor  
        >;  
};
```

Components which create groups of execution agents may use *execution categories* to communicate the forward progress and ordering guarantees of these execution agents with respect to other agents within the same group.

TODO: The meanings and relative "strength" of these categories are to be defined. Most of the wording for `sequenced_execution_tag`, `parallel_execution_tag`, and `unsequenced_execution_tag` can be migrated from S 25.2.3 p2, p3, and p4, respectively.

2.4.2 Associated shape type

```
template<class Executor>
struct executor_shape
{
    private:
        // exposition only
        template<class T>
        using helper = typename T::shape_type;

    public:
        using type = std::experimental::detected_or_t<
            size_t, helper, Executor
        >;

        // exposition only
        static_assert(std::is_integral_v<type>, "shape type must be an integral type");
};
```

2.4.3 Associated index type

```
template<class Executor>
struct executor_index
{
    private:
        // exposition only
        template<class T>
        using helper = typename T::index_type;

    public:
        using type = std::experimental::detected_or_t<
            executor_shape_t<Executor>, helper, Executor
        >;

        // exposition only
        static_assert(std::is_integral_v<type>, "index type must be an integral type");
};
```

2.5 Executor work guard

```
template<class Executor>
class executor_work_guard
{
    public:
        // types:

        typedef Executor executor_type;

        // construct / copy / destroy:

        explicit executor_work_guard(const executor_type& ex) noexcept;
        executor_work_guard(const executor_work_guard& other) noexcept;
        executor_work_guard(executor_work_guard&& other) noexcept;

        executor_work_guard& operator=(const executor_work_guard&) = delete;

        ~executor_work_guard();

        // executor work guard observers:

        executor_type get_executor() const noexcept;
        bool owns_work() const noexcept;

        // executor work guard modifiers:

        void reset() noexcept;

    private:
        Executor ex_; // exposition only
        bool owns_; // exposition only
};
```

2.5.1 Members

```
explicit executor_work_guard(const executor_type& ex) noexcept;
```

Effects: Initializes `ex_` with `ex`, and `owns_` with the result of `ex_.on_work_started()`.

Postconditions: `ex_ == ex`.

```
executor_work_guard(const executor_work_guard& other) noexcept;
```

Effects: Initializes `ex_` with `other.ex_`. If `other.owns_ == true`, initializes `owns_` with the result of `ex_.on_work_started()`; otherwise, sets `owns_` to false.

Postconditions: `ex_ == other.ex_`.

```
executor_work_guard(executor_work_guard&& other) noexcept;
```

Effects: Initializes `ex_` with `std::move(other.ex_)` and `owns_` with `other.owns_`, and sets `other.owns_` to false.

```
~executor_work_guard();
```

Effects: If `owns_` is true, performs `ex_.on_work_finished()`.

```
executor_type get_executor() const noexcept;
```

Returns: `ex_`.

```
bool owns_work() const noexcept;
```

Returns: `owns_`.

```
void reset() noexcept;
```

Effects: If `owns_` is true, performs `ex_.on_work_finished()`.

Postconditions: `owns_ == false`.

2.6 Polymorphic executor wrappers

2.6.1 General requirements on polymorphic executor wrappers

Polymorphic executors defined in this Technical Specification satisfy the `BaseExecutor`, `DefaultConstructible` (C++Std [defaultconstructible]), and `CopyAssignable` (C++Std [copyassignable]) requirements, and are defined as follows.

```
class C
{
public:
    class context_type; // TODO define this

    // construct / copy / destroy:

    C() noexcept;
    C(nullptr_t) noexcept;
    C(const executor& e) noexcept;
    C(executor&& e) noexcept;
    template<class Executor> C(Executor e);
    template<class Executor, class ProtoAllocator>
        C(allocator_arg_t, const ProtoAllocator& a, Executor e);

    C& operator=(const C& e) noexcept;
    C& operator=(C&& e) noexcept;
    C& operator=(nullptr_t) noexcept;
    template<class Executor> C& operator=(Executor e);

    ~C();

    // polymorphic executor modifiers:

    void swap(C& other) noexcept;
    template<class Executor, class ProtoAllocator>
        void assign(Executor e, const ProtoAllocator& a);

    // C operations:

    context_type context() const noexcept;

    // polymorphic executor capacity:

    explicit operator bool() const noexcept;
```

```

// polymorphic executor target access:

const type_info& target_type() const noexcept;
template<class Executor> Executor* target() noexcept;
template<class Executor> const Executor* target() const noexcept;
};

// polymorphic executor comparisons:

bool operator==(const C& a, const C& b) noexcept;
bool operator==(const C& e, nullptr_t) noexcept;
bool operator==(nullptr_t, const C& e) noexcept;
bool operator!=(const C& a, const C& b) noexcept;
bool operator!=(const C& e, nullptr_t) noexcept;
bool operator!=(nullptr_t, const C& e) noexcept;

// executor specialized algorithms:

void swap(C& a, C& b) noexcept;

// in namespace std:

template<class Allocator>
struct uses_allocator<C, Allocator>
    : true_type {};

```

[*Note:* To meet the `noexcept` requirements for executor copy constructors and move constructors, implementations may share a target between two or more executor objects. *--end note*]

The *target* is the executor object that is held by the wrapper.

2.6.1.1 Polymorphic executor constructors

```
C() noexcept;
```

Postconditions: `!*this`.

```
C(nullptr_t) noexcept;
```

Postconditions: `!*this`.

```
C(const C& e) noexcept;
```

Postconditions: `!*this` if `!e`; otherwise, `*this` targets `e.target()` or a copy of `e.target()`.

```
C(C&& e) noexcept;
```

Effects: If `!e`, `*this` has no target; otherwise, moves `e.target()` or move-constructs the target of `e` into the target of `*this`, leaving `e` in a valid state with an unspecified value.

```
template<class Executor> C(Executor e);
```

Effects: `*this` targets a copy of `e` initialized with `std::move(e)`.

```
template<class Executor, class ProtoAllocator>
C(allocator_arg_t, const ProtoAllocator& a, Executor e);
```

Effects: `*this` targets a copy of `e` initialized with `std::move(e)`.

A copy of the allocator argument is used to allocate memory, if necessary, for the internal data structures of the constructed `c` object.

2.6.1.2 Polymorphic executor assignment

```
C& operator=(const C& e) noexcept;
```

Effects: `C(e).swap(*this)`.

Returns: `*this`.

```
C& operator=(C&& e) noexcept;
```

Effects: Replaces the target of `*this` with the target of `e`, leaving `e` in a valid state with an unspecified value.

Returns: *this.

```
C& operator=(nullptr_t) noexcept;
```

Effects: C(nullptr).swap(*this).

Returns: *this.

```
template<class Executor> C& operator=(Executor e);
```

Effects: C(std::move(e)).swap(*this).

Returns: *this.

2.6.1.3 Polymorphic executor destructor

```
~C();
```

Effects: If *this != nullptr, releases shared ownership of, or destroys, the target of *this.

2.6.1.4 Polymorphic executor modifiers

```
void swap(C& other) noexcept;
```

Effects: Interchanges the targets of *this and other.

```
template<class Executor, class ProtoAllocator>
void assign(Executor e, const ProtoAllocator& a);
```

Effects: C(allocator_arg, a, std::move(e)).swap(*this).

2.6.1.5 Polymorphic executor operations

```
context_type context() const noexcept;
```

Requires: *this != nullptr.

Returns: A polymorphic wrapper for e.context(), where e is the target object of *this.

2.6.1.6 Polymorphic executor capacity

```
explicit operator bool() const noexcept;
```

Returns: true if *this has a target, otherwise false.

2.6.1.7 Polymorphic executor target access

```
const type_info& target_type() const noexcept;
```

Returns: If *this has a target of type T, typeid(T); otherwise, typeid(void).

```
template<class Executor> Executor* target() noexcept;
template<class Executor> const Executor* target() const noexcept;
```

Returns: If target_type() == typeid(Executor) a pointer to the stored executor target; otherwise a null pointer value.

2.6.1.8 Polymorphic executor comparisons

```
bool operator==(const C& a, const C& b) noexcept;
```

Returns:

- true if !a and !b;
- true if a and b share a target;
- true if e and f are the same type and e == f, where e is the target of a and f is the target of b;
- otherwise false.

```
bool operator==(const C& e, nullptr_t) noexcept;
bool operator==(nullptr_t, const C& e) noexcept;
```

Returns: !e.

```
bool operator!=(const C& a, const C& b) noexcept;
```

Returns: !(a == b).

```
bool operator!=(const C& e, nullptr_t) noexcept;
bool operator!=(nullptr_t, const C& e) noexcept;
```

Returns: (bool) e.

2.6.1.9 Polymorphic executor specialized algorithms

```
void swap(C& a, C& b) noexcept;
```

Effects: a.swap(b).

2.6.2 Class one_way_executor

Class one_way_executor satisfies the general requirements on polymorphic executor wrappers, with the additional definitions below.

```
class one_way_executor
{
public:
    // execution agent creation
    template<class Function, class ProtoAllocator = std::allocator<void>>
        void execute(Function&& f, const ProtoAllocator& a = ProtoAllocator()) const;
};
```

Class one_way_executor satisfies the OneWayExecutor requirements. The target object shall satisfy the OneWayExecutor requirements.

```
template<class Function, class ProtoAllocator>
    void execute(Function&& f, const ProtoAllocator& a) const;
```

Let *e* be the target object of *this. Let *a1* be the allocator that was specified when the target was set. Let *fd* be the result of DECAY_COPY(std::forward<Function>(f)).

Effects: Performs e.execute(g, a1), where *g* is a function object of unspecified type that, when called as *g()*, performs *fd()*. The allocator *a* is used to allocate any memory required to implement *g*.

2.6.3 Class non_blocking_one_way_executor

Class non_blocking_one_way_executor satisfies the general requirements on polymorphic executor wrappers, with the additional definitions below.

```
class non_blocking_one_way_executor
{
public:
    // execution agent creation
    template<class Function, class ProtoAllocator = std::allocator<void>>
        void execute(Function&& f, const ProtoAllocator& a = ProtoAllocator()) const;
    template<class Function, class ProtoAllocator = std::allocator<void>>
        void post(Function&& f, const ProtoAllocator& a = ProtoAllocator()) const;
    template<class Function, class ProtoAllocator = std::allocator<void>>
        void defer(Function&& f, const ProtoAllocator& a = ProtoAllocator()) const;
};
```

Class non_blocking_one_way_executor satisfies the NonBlockingOneWayExecutor requirements. The target object shall satisfy the NonBlockingOneWayExecutor requirements.

```
template<class Function, class ProtoAllocator>
    void execute(Function&& f, const ProtoAllocator& a) const;
```

Let *e* be the target object of *this. Let *a1* be the allocator that was specified when the target was set. Let *fd* be the result of DECAY_COPY(std::forward<Function>(f)).

Effects: Performs e.execute(g, a1), where *g* is a function object of unspecified type that, when called as *g()*, performs *fd()*. The allocator *a* is used to allocate any memory required to implement *g*.

```
template<class Function, class ProtoAllocator>
    void post(Function&& f, const ProtoAllocator& a) const;
```

Let *e* be the target object of *this. Let *a1* be the allocator that was specified when the target was set. Let *fd* be the result of DECAY_COPY(std::forward<Function>(f)).

Effects: Performs `e.post(g, a1)`, where `g` is a function object of unspecified type that, when called as `g()`, performs `fd()`. The allocator `a` is used to allocate any memory required to implement `g`.

```
template<class Function, class ProtoAllocator>
void defer(Function&& f, const ProtoAllocator& a) const;
```

Let `e` be the target object of `*this`. Let `a1` be the allocator that was specified when the target was set. Let `fd` be the result of `DECAY_COPY(std::forward<Function>(f))`.

Effects: Performs `e.defer(g, a1)`, where `g` is a function object of unspecified type that, when called as `g()`, performs `fd()`. The allocator `a` is used to allocate any memory required to implement `g`.

2.6.4 Class `two_way_executor`

Class `two_way_executor` satisfies the general requirements on polymorphic executor wrappers, with the additional definitions below.

```
class two_way_executor
{
public:
    // execution agent creation
    template<class Function, class ProtoAllocator = std::allocator<void>>
        result_of_t<decay_t<Function>()>
        sync_execute(Function&& f, const ProtoAllocator& a = ProtoAllocator()) const;
    template<class Function, class ProtoAllocator = std::allocator<void>>
        std::future<result_of_t<decay_t<Function>()>>
        async_execute(Function&& f, const ProtoAllocator& a = ProtoAllocator()) const;
};
```

Class `two_way_executor` satisfies the `TwoWayExecutor` requirements. The target object shall satisfy the `TwoWayExecutor` requirements.

```
template<class Function, class ProtoAllocator>
result_of_t<decay_t<Function>()>
sync_execute(Function&& f, const ProtoAllocator& a);
```

Let `e` be the target object of `*this`. Let `a1` be the allocator that was specified when the target was set. Let `fd` be the result of `DECAY_COPY(std::forward<Function>(f))`.

Effects: Performs `e.sync_execute(g, a1)`, where `g` is a function object of unspecified type that, when called as `g()`, performs `fd()`. The allocator `a` is used to allocate any memory required to implement `g`.

Returns: The return value of `fd()`.

```
template<class Function, class ProtoAllocator>
std::future<result_of_t<decay_t<Function>()>>
async_execute(Function&& f, const ProtoAllocator& a) const;
```

Let `e` be the target object of `*this`. Let `a1` be the allocator that was specified when the target was set. Let `fd` be the result of `DECAY_COPY(std::forward<Function>(f))`.

Effects: Performs `e.async_execute(g, a1)`, where `g` is a function object of unspecified type that, when called as `g()`, performs `fd()`. The allocator `a` is used to allocate any memory required to implement `g`.

Returns: A future with an associated shared state that will contain the result of `fd()`. [*Note:* `e.async_execute(g)` may return any future type that satisfies the `Future` requirements, and not necessarily `std::future`. One possible implementation approach is for the polymorphic wrapper to attach a continuation to the inner future via that object's `then()` member function. When invoked, this continuation stores the result in the outer future's associated shared and makes that shared state ready. --end note]

2.7 Thread pools

Thread pools create execution agents which execute on threads without incurring the overhead of thread creation and destruction whenever such agents are needed.

2.7.1 Header `<thread_pool>` synopsis

```
namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    class static_thread_pool;
```



```

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std

```

2.7.2 Class static_thread_pool

`static_thread_pool` is a statically-sized thread pool which may be explicitly grown via thread attachment. However, `static_thread_pool` does not automatically change size.

`static_thread_pool` presents an effectively unbounded input queue and the execution functions of `static_thread_pool`'s associated executors do not block on this input queue.

[*Note:* Because `static_thread_pool` provides parallel execution agents, situations which require concurrent execution properties are not guaranteed correctness. --*end note.*]

```

class static_thread_pool
{
public:
    class executor_type;

    // construction/destruction
    explicit static_thread_pool(std::size_t num_threads);

    // nocopy
    static_thread_pool(const static_thread_pool&) = delete;
    static_thread_pool& operator=(const static_thread_pool&) = delete;

    // stop accepting incoming work and wait for work to drain
    ~static_thread_pool();

    // attach current thread to the thread pools list of worker threads
    void attach();

    // signal all work to complete
    void stop();

    // wait for all threads in the thread pool to complete
    void wait();

    // placeholder for a general approach to getting executors from
    // standard contexts.
    executor_type executor() noexcept;
};

bool operator==(const static_thread_pool& a, const static_thread_pool& b) noexcept;
bool operator!=(const static_thread_pool& a, const static_thread_pool& b) noexcept;

```

The class `static_thread_pool` satisfies the `ExecutionContext` requirements.

For an object of type `static_thread_pool`, *outstanding work* is defined as the sum of:

- the total number of calls to the `on_work_started` function that returned `true`, less the total number of calls to the `on_work_finished` function, on any executor associated with the `static_thread_pool`.
- the number of function objects that have been added to the `static_thread_pool` via the `static_thread_pool` executor, but not yet executed; and
- the number of function objects that are currently being executed by the `static_thread_pool`.

The `static_thread_pool` member functions `executor`, `attach`, `wait`, and `stop`, and the `static_thread_pool::executor_type` copy constructors and member functions, do not introduce data races as a result of concurrent calls to those functions from different threads of execution.

2.7.2.1 Construction and destruction

```
static_thread_pool(std::size_t num_threads);
```

Effects: Constructs a `static_thread_pool` object with `num_threads` threads of execution, as if by creating objects of type `std::thread`.

```
~static_thread_pool();
```

Effects: Destroys an object of class `static_thread_pool`. Performs `stop()` followed by `wait()`.

2.7.2.2 Worker Management

```
void attach();
```

Effects: adds the calling thread to the pool of workers. Blocks the calling thread until signalled to complete by `stop()` or `wait()`, and then blocks until all the threads created during `static_thread_pool` object construction have completed. (Note: The implementation is required to use the attached thread to execute submitted function objects. RATIONALE: implementations in terms of the Windows thread pool cannot utilise user-provided threads. --end note) (NAMING: a possible alternate name for this function is `join()`.)

```
void stop();
```

Effects: Signals the threads in the pool to complete as soon as possible. If a thread is currently executing a function object, the thread will exit only after completion of that function object. The call to `stop()` returns without waiting for the threads to complete. Subsequent calls to `attach` complete immediately.

```
void wait();
```

Effects: If not already stopped, signals the threads in the pool to complete once the outstanding work is 0. Blocks the calling thread (C++Std [defns.block]) until all threads in the pool have completed, without executing submitted function objects in the calling thread. Subsequent calls to `attach` complete immediately.

Synchronization: The completion of each thread in the pool synchronizes with (C++Std [intro.multithread]) the corresponding successful `wait()` return.

2.7.2.3 Executor Creation

```
executor_type executor() noexcept;
```

Returns: An executor that may be used to submit function objects to the thread pool.

2.7.2.4 Comparisons

```
bool operator==(const static_thread_pool& a, const static_thread_pool& b) noexcept;
```

Returns: `std::addressof(a) == std::addressof(b)`.

```
bool operator!=(const static_thread_pool& a, const static_thread_pool& b) noexcept;
```

Returns: `!(a == b)`.

2.7.3 Class `static_thread_pool::executor_type`

```
class static_thread_pool::executor_type
{
public:
    // types:

    typedef parallel_execution_tag execution_category;
    typedef possibly_blocking_execution_tag blocking_category;
    typedef std::size_t shape_type;
    typedef std::size_t index_type;

    // construct / copy / destroy:

    executor_type(const executor_type& other) noexcept;
    executor_type(executor_type&& other) noexcept;

    executor_type& operator=(const executor_type& other) noexcept;
    executor_type& operator=(executor_type&& other) noexcept;

    // executor operations:

    bool running_in_this_thread() const noexcept;

    static_thread_pool& context() const noexcept;

    bool on_work_started() const noexcept;
    void on_work_finished() const noexcept;
};

bool operator==(const static_thread_pool::executor_type& a,
                const static_thread_pool::executor_type& b) noexcept;
```

```
bool operator!=(const static_thread_pool::executor_type& a,  
               const static_thread_pool::executor_type& b) noexcept;
```

`static_thread_pool::executor_type` is a type satisfying the `BaseExecutor` and `ExecutorWorkTracker` requirements. Objects of type `static_thread_pool::executor` are associated with a `static_thread_pool`.

The customization points `execute`, `post`, `defer`, `sync_execute`, `async_execute`, `async_post`, `async_defer`, `then_execute`, `bulk_execute`, `bulk_post`, `bulk_defer`, `bulk_sync_execute`, `bulk_async_execute`, `bulk_async_post`, `bulk_async_defer`, and `bulk_then_execute` are well-formed for this executor. Function objects submitted using these customization points will be executed by the `static_thread_pool`.

For the customization points `execute`, `sync_execute`, `async_execute`, `bulk_execute`, `bulk_sync_execute`, and `bulk_async_execute`, if `running_in_this_thread()` is `true`, calls at least one of the submitted function objects in the current thread prior to returning from the customization point. [*Note: If this function object exits via an exception, the exception propagates to the caller. --end note*]

2.7.3.1 Constructors

```
executor_type(const executor_type& other) noexcept;
```

Postconditions: `*this == other`.

```
executor_type(executor_type&& other) noexcept;
```

Postconditions: `*this` is equal to the prior value of `other`.

2.7.3.2 Assignment

```
executor_type& operator=(const executor_type& other) noexcept;
```

Postconditions: `*this == other`.

Returns: `*this`.

```
executor_type& operator=(executor_type&& other) noexcept;
```

Postconditions: `*this` is equal to the prior value of `other`.

Returns: `*this`.

2.7.3.3 Operations

```
bool running_in_this_thread() const noexcept;
```

Returns: `true` if the current thread of execution is a thread that was created by or attached to the associated `static_thread_pool` object.

```
static_thread_pool& context() const noexcept;
```

Returns: A reference to the associated `static_thread_pool` object.

```
bool on_work_started() const noexcept;
```

Effects: Increments the count of outstanding work associated with the `static_thread_pool`.

Returns: `false` if there was a prior call to the `stop()` member function of the associated `static_thread_pool` object; otherwise `true`.

```
void on_work_finished() const noexcept;
```

Effects: Decrements the count of outstanding work associated with the `static_thread_pool`.

2.7.3.4 Comparisons

```
bool operator==(const static_thread_pool::executor_type& a,  
               const static_thread_pool::executor_type& b) noexcept;
```

Returns: `a.context() == b.context()`.

```
bool operator!=(const static_thread_pool::executor_type& a,  
               const static_thread_pool::executor_type& b) noexcept;
```

Returns: `!(a == b)`.

2.8 Interoperation with existing facilities

2.8.1 Execution policy interoperation

```
class parallel_execution_policy
{
public:
    // types:
    using execution_category = parallel_execution_tag;
    using executor_type = implementation-defined;

    // executor access
    const executor_type& executor() const noexcept;

    // execution policy factory
    template<class Executor>
    see-below on(Executor&& exec) const;
};

class sequenced_execution_tag { by-analogy-to-parallel_execution_policy };
class parallel_unsequenced_execution_tag { by-analogy-to-parallel_execution_policy };
```

2.8.1.1 Associated executor

Each execution policy is associated with an executor, and this executor is called its *associated executor*.

The type of an execution policy's associated executor shall satisfy the requirements of `BulkTwoWayExecutor`.

When an execution policy is used as a parameter to a parallel algorithm, the execution agents that invoke element access functions are created by the execution policy's associated executor.

The type of an execution policy's associated executor is the member type `executor_type`.

2.8.1.2 Execution category

Each execution policy is categorized by an *execution category*.

When an execution policy is used as a parameter to a parallel algorithm, the execution agents it creates are guaranteed to make forward progress and execute invocations of element access functions as ordered by its execution category.

An execution policy's execution category is given by the member type `execution_category`.

The execution category of an execution policy's associated executor shall not be weaker than the execution policy's execution category.

2.8.1.3 Associated executor access

```
const executor_type& executor() const noexcept;
```

Returns: The execution policy's associated executor.

2.8.1.4 Execution policy factory

```
template<class Executor>
see-below on(Executor&& exec) const;
```

Let `T` be `decay_t<Executor>`.

Returns: An execution policy whose execution category is `execution_category`. If `T` satisfies the requirements of `BulkTwoWayExecutor`, the returned execution policy's associated executor is equal to `exec`. Otherwise, the returned execution policy's associated executor fulfills the `BulkTwoWayExecutor` requirements which creates execution agents using a copy of `exec`.

Remarks: This member function shall not participate in overload resolution unless `is_executor_v<T>` is true and `executor_execution_category_t<T>` is as strong as `execution_category`.

2.8.2 Control structure interoperation

2.8.2.1 Function template `async`

The function template `async` provides a mechanism to invoke a function in a new execution agent created by an executor and provides the result of the function in the future object with which it shares a state.

```
template<class Executor, class Function, class... Args>
executor_future_t<Executor, result_of_t<decay_t<Function>(decay_t<Args>...)>>
async(const Executor& exec, Function&& f, Args&&... args);
```

Returns: Equivalent to:

```
auto __g = bind(std::forward<Function>(f), std::forward<Args>(args)...);
return execution::async_post(exec, [&__g = move(__g)]{ return INVOKE(__g); });
```

2.8.2.2 `std::experimental::future::then()`

The member function template `then` provides a mechanism for attaching a *continuation* to a `std::future` object, which will be executed on a new execution agent created by an executor.

```
template<class T>
template<class Executor, class Function>
executor_future_t<Executor, see-below>
future<T>::then(const Executor& exec, Function&& f);
```

2. TODO: Concrete specification

The general idea of this overload of `.then()` is that it accepts a particular type of `OneWayExecutor` that cannot block in `.execute()`. `.then()` stores `f` as the next continuation in the future state, and when the future is ready, creates an execution agent using a copy of `exec`.

One approach is for `.then()` to require a `NonBlockingOneWayExecutor`, and to specify that `.then()` submits the continuation using `exec.post()` if the future is already ready at the time when `.then()` is called, and to submit using `exec.execute()` otherwise.

2.8.2.3 `std::experimental::shared_future::then()`

The member function template `then` provides a mechanism for attaching a *continuation* to a `std::shared_future` object, which will be executed on a new execution agent created by an executor.

```
template<class T>
template<class Executor, class Function>
executor_future_t<Executor, see-below>
shared_future<T>::then(const Executor& exec, Function&& f);
```

TODO: Concrete specification

The general idea of this overload of `.then()` is that it accepts a particular type of `OneWayExecutor` that cannot block in `.execute()`. `.then()` stores `f` as the next continuation in the underlying future state, and when the underlying future is ready, creates an execution agent using a copy of `exec`.

One approach is for `.then()` to require a `NonBlockingOneWayExecutor`, and to specify that `.then()` submits the continuation using `exec.post()` if the future is already ready at the time when `.then()` is called, and to submit using `exec.execute()` otherwise.

2.8.2.4 Function template `invoke`

The function template `invoke` provides a mechanism to invoke a function in a new execution agent created by an executor and return result of the function.

```
template<class Executor, class Function, class... Args>
result_of_t<F&&(Args&&...)>
invoke(const Executor& exec, Function&& f, Args&&... args);
```

Returns: Equivalent to:

```
return execution::sync_execute(exec, [&]{ return INVOKE(f, args...); });
```

2.8.2.5 Task block

2.8.2.5.1 Function template `define_task_block_restore_thread()`

```
template<class Executor, class F>
void define_task_block_restore_thread(const Executor& exec, F&& f);
```

Requires: Given an lvalue `tb` of type `task_block`, the expression `f(tb)` shall be well-formed.

Effects: Constructs a `task_block` `tb`, creates a new execution agent, and calls `f(tb)` on that execution agent.

Throws: `exception_list`, as specified in version two of the Paralellism TS.

Postconditions: All tasks spawned from `f` have finished execution.

Remarks: Unlike `define_task_block`, `define_task_block_restore_thread` always returns on the same thread as the one on which it was called.

2.8.2.5.2 `task_block` member function template `run`

```
template<class Executor, class F>
void run(const Executor& exec, F&& f);
```

Requires: `F` shall be `MoveConstructible`. `DECAY_COPY(std::forward<F>(f))()` shall be a valid expression.

Preconditions: `*this` shall be an active `task_block`.

Effects: Evaluates `DECAY_COPY(std::forward<F>(f))()`, where `DECAY_COPY(std::forward<F>(f))` is evaluated synchronously within the current thread. The call to the resulting copy of the function object is permitted to run on an execution agent created by `exec` in an unordered fashion relative to the sequence of operations following the call to `run(exec, f)` (the continuation), or indeterminately-sequenced within the same thread as the continuation. The call to `run` synchronizes with the next invocation of `wait` on the same `task_block` or completion of the nearest enclosing `task_block` (i.e., the `define_task_block` or `define_task_block_restore_thread` that created this `task_block`.

Throws: `task_cancelled_exception`, as described in version 2 of the Parallelism TS.

3 Relationship to other proposals and specifications

3.1 Networking TS

Executors in the Networking TS may be defined as refinements of the type requirements in this proposal, as illustrated below. In addition to these requirements, some minor changes would be required to member function names and parameters used in the Networking TS, to conform to the requirements defined in this proposal.

3.1.1 `NetworkingExecutor` requirements

A type `X` satisfies the `NetworkingExecutor` requirements if it satisfies the `NonBlockingOneWayExecutor` requirements, the `ExecutorWorkTracker` requirements, and satisfies the additional requirements listed below.

In the Table below, `x` denotes a (possibly `const`) value of type `x`.

expression	return type	assertion/note pre/post-condition
<code>x.context()</code>	<code>net::execution_context&</code> , or <code>E&</code> where <code>E</code> is a type that satisfies the <code>NetworkingExecutionContext</code> requirements.	

3.1.2 `NetworkingExecutionContext` requirements

A type `X` satisfies the `NetworkingExecutionContext` requirements if it satisfies the `ExecutionContext` requirements, is publicly and unambiguously derived from `net::execution_context`, and satisfies the additional requirements listed below.

In the Table below, `x` denotes a value of type `x`.

(NetworkingExecutionContext requirements)		
expression	return type	assertion/note pre/post-condition
<code>X::executor_type</code>	type meeting <code>NetworkingExecutor</code> requirements	
<code>x.~X()</code>		Destroys all unexecuted function objects that were submitted via an executor object that is associated with the execution context.
<code>x.get_executor()</code>	<code>X::executor_type</code>	Returns an executor object that is associated with the execution context.

4 Appendix

4.1 Code Examples

These short code examples demonstrate how we expect programmers to use executors to create execution in different use cases.

4.1.1 Simple use of an executor

A programmer may create an asynchronous task via `async()` by providing an executor obtained from a thread pool:

```
using namespace std::experimental::concurrency_v2;

// create a thread pool object
static_thread_pool pool(4);

// obtain an executor from the thread pool
auto exec = pool.executor()

auto my_task = ...

// compose the thread pool's executor with async:
async(exec, my_task);
```

In this example, the executor parameter provides `std::async()` with explicit requirements concerning how to create the work responsible for executing the task.

Similarly, a programmer may require that the work created by a parallel algorithm happen "on" an executor:

```
using namespace std::experimental::concurrency_v2;

// create a thread pool object
static_thread_pool pool(4);

// obtain an executor from the thread pool
auto exec = pool.executor()

auto my_task = ...

// compose the thread pool's executor with the par execution policy using .on():
for_each(execution::par.on(exec), vec.begin(), vec.end(), my_task);
```

In this example, the executor parameter composes with `par` to create a new execution policy associated with the thread pool's executor.

4.1.2 Use of an executor in a generic context

Functions may receive executors as generic template parameters. Our proposal provides tools for working in generic contexts while retaining the ability to manipulate executors uniformly. For example, consider this implementation of a hypothetical math library's `parallel dot()` product function:

```
template<class BulkTwoWayExecutor>
float dot(const BulkTwoWayExecutor& exec, const float* data1, const float* data2, size_t n)
{
    auto zipped = zip_with(std::multiplies<>(), data1, data2);
    return std::reduce(std::execution::par.on(exec), zipped, zipped + n, 0.f, std::plus<>());
}
```

Because it is a template, the `dot()` function has no concrete information about `exec`, besides the fact that the executor satisfies the requirements of our `BulkTwoWayExecutor` concept. However, this is all the information necessary to compose `exec` correctly with `par` and `std::reduce`.

Programmers may work at a lower level yet retain generality by manipulating an executor through customization points. For example, consider this hypothetical implementation of `async()`:

```
template<class Executor, class Function, class... Args>
auto async(const Executor& exec, Function&& f, Args&&... args)
{
    // bind f and args together
    auto g = bind(f, args...);
```

```
using namespace std::experimental::concurrency_v2;

// use the customization point execution::async_execute() to asynchronously invoke g and return a future
return execution::async_execute(exec, g);
}
```

Even if the executor in this example does not natively support the `async_execute()` operation through a member or free function, the executor may still be used as if it does support this operation. If native support does not exist, the customization point applies an adaptation to whatever native operations do exist to create the desired operation.

4.1.3 Defining executors

A programmer creates an executor by defining a type with one or more execution functions defined as members or free functions as well as other members related to executor identity.

For example, an executor which creates a new thread for each execution agent may define the `execute()` method:

```
struct per_thread_executor
{
    template<class Function>
    void execute(Function&& f) const
    {
        std::thread new_thread(std::forward<Function>(f));

        new_thread.detach();
    }

    const per_thread_executor& context() const noexcept
    {
        return *this;
    }

    bool operator==(const per_thread_executor&) const noexcept
    {
        return true;
    }

    bool operator!=(const per_thread_executor&) const noexcept
    {
        return false;
    }
};
```

An executor which executes work immediately within the calling thread may define the `sync_execute()` method:

```
struct inline_executor
{
    template<class Function>
    auto sync_execute(Function&& f) const
    {
        return std::forward<Function>(f)();
    }

    const inline_executor& context() const noexcept
    {
        return *this;
    }

    bool operator==(const inline_executor&) const noexcept
    {
        return true;
    }

    bool operator!=(const inline_executor&) const noexcept
    {
        return false;
    }
};
```

An executor which executes parallel work in bulk using OpenMP may define the `bulk_sync_execute()` method:

```
struct openmp_executor
{
    using execution_category = parallel_execution_tag;

    template<class Function, class ResultFactory, class SharedFactory>
    auto bulk_sync_execute(Function f, size_t n, ResultFactory result_factory, SharedFactory shared_factory) const
```



```
{
    auto result = result_factory();
    auto shared = shared_factory();

    #pragma omp parallel for
    for(size_t i = 0; i < n; ++i)
    {
        f(i, result, shared);
    }

    return result;
}

const openmp_executor& context() const noexcept
{
    return *this;
}

bool operator==(const openmp_executor&) const noexcept
{
    return true;
}

bool operator!=(const openmp_executor&) const noexcept
{
    return false;
}
};
```